Part 2
파이썬으로 데이터 분석하기

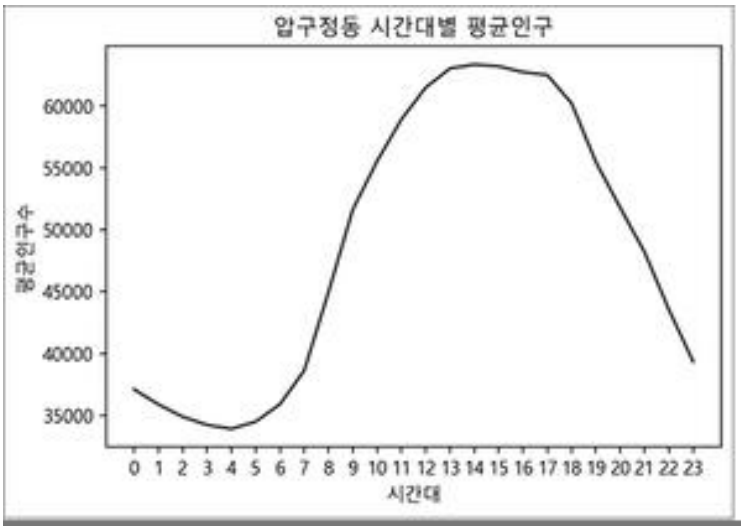
8장 그래프로 데이터 시각화하기
9장 데이터 분석 기초
10장 프로젝트 로드맵 그리기
11장 프로젝트로 파이썬 완성하기: 핫플레이스 인구 분석
12장 위치 정보 시각화하기: folium

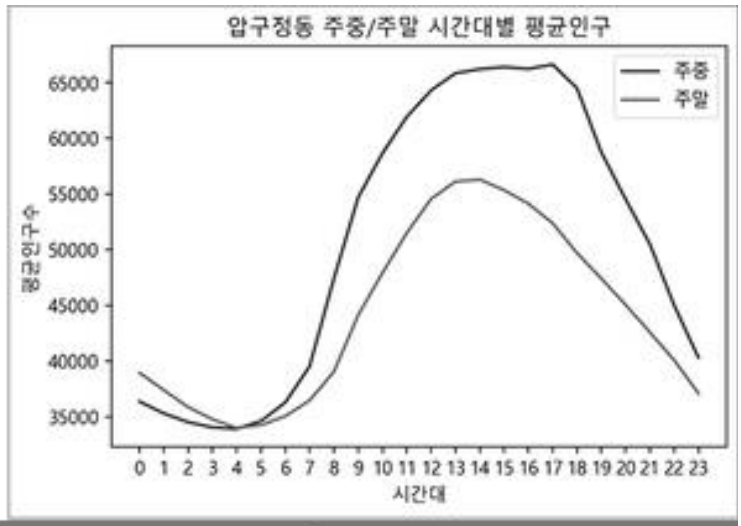
지금까지 파이썬 프로그래밍의 기본이 되는 개념을 배웠습니다. 입출력 명령어부터 3가지 제어구조, 여러 데이터를 한 번에 저장하는 리스트와 딕셔너리까지 많은 것을 배웠어요. 이제는 이렇게 배운 내용을 바탕으로 무언가를 해볼 때입니다.

파이썬으로 할 수 있는 일은 많습니다. 웹 페이지를 만들 수도 있고, 웹에서 필요한 정보를 자동으로 긁어모으는 웹 크롤링이나 업무 자동화를 할 수 있습니다. 또한, 애플리케이션이나 게임을 개발할 수도 있죠. 그중에서도 이 책에서 도전해 볼 과제는 '데이터 분석'입니다. 파이썬은 데이터를 수집하고 분석하기 쉽게 도와주는 다양한 도구를 지원해서 데이터 분석에 활용하기 좋습니다. 데이터만 있으면 원하는 대로 이리저리 분석할 수 있고 사전에 알아야 하는 프로그래밍 지식도 Part 1에서 배운 정도면 충분합니다. 물론 데이터 분석에 필요한 라이브러리 사용법은 더 알아야 하지만요. 그래도 순서대로 잘 따라온다면 무리 없이 프로젝트를 완성할 수 있습니다. 그럼 동기 유발을 위해 이 책에서 배울 프로젝트를 미리 훑어볼까요?

휴일이나 주말에 친구 또는 가족과 맛집이나 재밌는 장소에 가고 싶을 때가 있죠. 그런데 사람들이 너무 많을까봐 걱정된 적은 없었나요? 그래서 이 책에서는 가고 싶은 지역의 인구 데이터를 분석하는 프로젝트를 진행합니다. 프로젝트를 완성하고 나면 서울시에서 원하는 행정동의 인구 그래프를 조회할 수 있습니다. 또한 프로젝트를 참고해 스스로 다른 도시의 인구도 분석할 수 있습니다. 그리고 배운 내용을 응용한다면 인구 데이터가 아닌 다른 데이터도 분석할 수 있습니다.

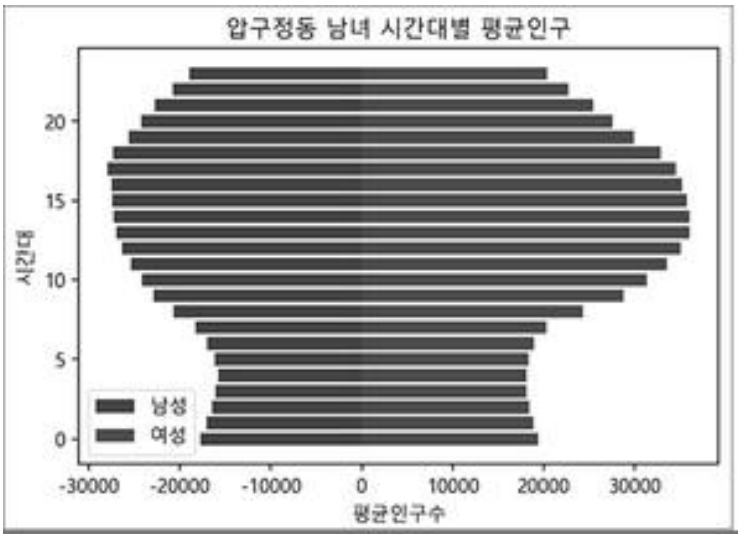
이 책에서는 핫플레이스가 있는 지역의 인구 데이터로 그래프를 그려 분석합니다. 코드를 작성해 다음과 같이 여러 형태의 그래프를 그려 보고 그 결과를 분석하는 것이죠.

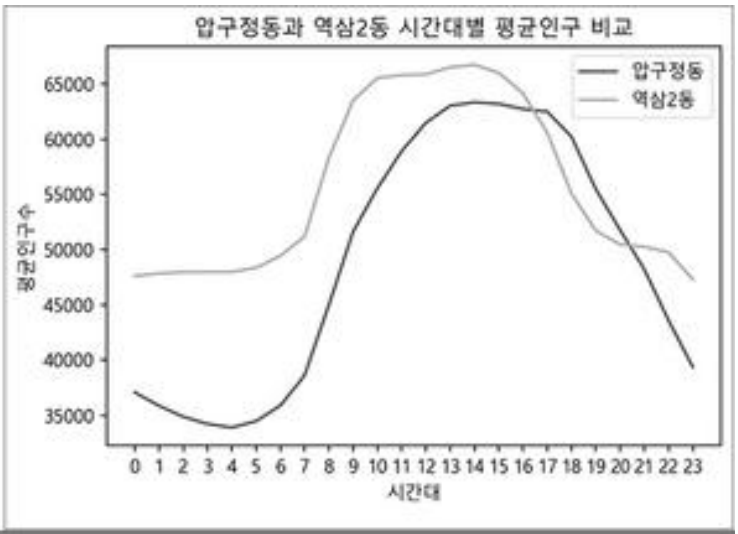




Copyright © Gilbut, Inc. All rights reserved.

시간대별 인구뿐만 아니라 성별 인구와 두 지역의 인구를 비교하기도 합니다.





Copyright © Gilbut, Inc. All rights reserved.

결과를 지도로도 나타내 봅니다.



배운 내용을 참고해 잘 응용한다면 여러분이 원하는 형태로도 나타낼 수 있을 거예요. 재밌을 것 같지 않나요? 그럼 데이터 분석 프로젝트에 필요한 몇 가지부터 배우고 프로젝트에 들어갑시다. Let's get it!

8장
크래프로 데이터 시각화하기

데이터를 이해하기 쉽게 시각적으로 표현하는 것을 데이터 시각화라고 합니다. 앞에서 살펴본 matplotlib 라이브러리도 데이터 시각화의 대표적인 예입니다. 시간대별 인구 데이터를 한눈에 볼 수 있도록 그래프로 표현한 것이 대표적입니다. 이 장에서는 데이터 분석에 들어가기 전에 데이터를 시각화하는 방법을 배워 보겠습니다.

8.1

그래프 그리기

8.1.1 데이터 시각화 도구 사용하기

데이터 시각화의 기본은 그래프 그리기입니다. 그림을 그릴 때 화구가 필요하듯이 그래프를 그리려면 도구가 필요합니다. 파이썬에는 `matplotlib`이라는 매우 강력한 도구가 있습니다. `matplotlib`은 데이터 시각화를 돕는 파이썬 라이브러리입니다.

모듈, 패키지, 라이브러리

1장에서 파이썬은 다양한 확장 기능을 구현해 놓은 라이브러리가 많다고 했죠. 5장에서는 라이브러리에 관해 잠깐 설명했습니다. 여기서 그 개념을 조금 더 짚고 넘어가겠습니다.

인터넷에서 파이썬 자료를 검색하다 보면 모듈, 패키지, 라이브러리라는 용어를 자주 접하게 됩니다. 이 셋은 비슷해 보이지만 조금씩 차이가 있습니다.

- 모듈(module) 특정 기능이 정의되어 있는 단일 파일
- 패키지(package) 여러 모듈을 모아 둔 것
- 라이브러리(library) 모듈과 패키지를 모아 둔 것

모듈은 특정 기능을 구현해 놓은 코드가 들어 있는 하나의 파일입니다. 패키지는 여러 문서를 한 폴더에 넣어 관리하듯이 여러 모듈을 하나로 묶은 것을 말합니다. 이런 모듈과 패키지를 묶어 라이브러리라고 합니다.

라이브러리에도 표준 라이브러리와 외부 라이브러리가 있습니다. 표준 라이브러리는 파이썬에서 공식적으로 제공하는 라이브러리로, 파이썬을 설치할 때 함께 설치됩니다. 표준에 포함되지 않은 외부 라이브러리는 파이썬을 설치할 때 자동으로 설치되지 않으므로 사용할 때 별도로 설치해야 합니다.

모듈, 패키지, 라이브러리는 혼용하는 경우가 많습니다. 5장에서 `random`을 라이브러리라고 했죠. 그러나 찾아보면 `random`을 모듈이라고 하기도 합니다. 이 장에서 배울 `matplotlib` 역시 패키지 또는 라이브러리로 혼용합니다. 따라서 처음부터 그 의미를 정확히 구분하려고 하기보다는 '특정 기능을 구현해 놓은 형태' 정도로 이해해도 괜찮습니다.

이 셋을 파이썬 파일 또는 프로그램에서 사용하려면 `import`로 추가해야 합니다. 5장에서 `import random`으로 `random` 라이브러리를 추가한 것처럼 패키지나 모듈도 추가해야 합니다.

NOTE help()로 정보 확인하기

파이썬에서 사용하는 명령어, 모듈, 패키지, 라이브러리 등의 정보를 알고 싶다면 `help()`를 활용해 보세요. `help()`는 괄호 안의 대상에 관한 정보를 알려 주는 명령어입니다. `help(random)`을 실행해 `random`에 대한 정보를 확인해 봅시다. 코드에서 보듯이 `help()`를 실행할 때는 작업하는 파일에서 읽을 수 있게 대상이 속한 모듈, 패키지, 라이브러리를 추가해야 합니다.

```
import random
help(random)
```

실행결과

```
Help on module random:

NAME
    random - Random variable generators.
…
FUNCTIONS
…
    randint(a, b) method of Random instance
        Return random integer in range [a, b], including both end points.
…
```

실행결과가 굉장히 길게 출력되는데, 몇 가지만 살펴봅시다. 첫 줄에 `Help on module random`을 보면 `random`을 모듈로 소개합니다. 스크롤을 내려 `FUNCTIONS` 항목을 봅시다. `FUCTIONS` 항목에서는 `random` 모듈에 속한 명령어를 보여 줍니다. 알파벳 순서로 나열되어 있는데, r 부분을 찾으면 앞에서 사용한 `randint()`가 있습니다. 설명을 보면 '범위 [a, b]에서 양 끝을 모두 포함하는 무작위 정수를 돌려준다'고 나옵니다. 모듈만이 아니라 특정 명령어에 대한 정보도 바로 확인할 수 있습니다. `help(random.randint)`처럼 직접 명령어를 넣으면 됩니다. 궁금한 명령어가 있으면 `help()`로 정보를 확인해 보세요.

이 장에서 사용할 `matplotlib`은 모듈, 패키지, 라이브러리 중에 무엇일까요?

```
import matplotlib
help(matplotlib)
```

```
실행결과
Help on package matplotlib:

NAME
    matplotlib - This is an object-oriented plotting library.
…
```

첫 줄을 보면 `matplotlib`은 패키지라고 나옵니다. 그런데 바로 아래 `NAME` 항목에는 'matplotlib은 그래프를 그리는 객체지향 라이브러리'라고 나옵니다. 여기서도 패키지와 라이브러리를 혼용하고 있습니다. 이 장에서는 `matplotlib` 중 `pyplot`을 주로 사용하니 `pyplot`도 살펴보죠.

```
import matplotlib
help(matplotlib.pyplot)
```

```
실행결과
Help on module matplotlib.pyplot in matplotlib:
…
```

`matplotlib` 안에 있는 모듈 `matplotlib.pyplot`이라고 나오는 걸로 보아 `pyplot`은 모듈임을 알 수 있습니다.

포함 관계를 나타내는 도트(.)

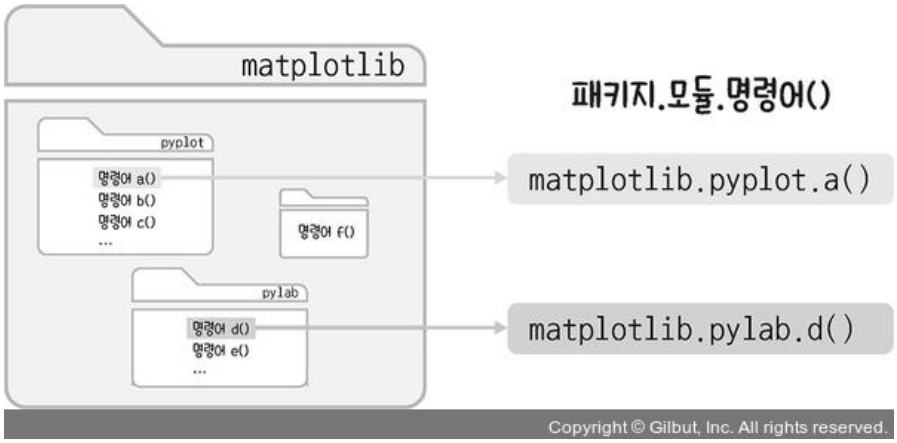
모듈, 패키지, 라이브러리를 배워 보니 포함 관계가 굉장히 많죠. 이 포함 관계를 표현하는 방법으로 파이썬에서는 도트(.)를 사용합니다. 도트(.)는 상위 개념에서 하위 개념으로 내려갈 때 `상위개념.하위개념` 형식으로 사용합니다.

`random` 모듈에는 다양한 명령어가 있는데, 대표적으로 `randint()`가 있습니다. 이를 나타내는 의미로 `randint()`를 사용할 때 `random.randint()`로 작성합니다.

`matplotlib` 패키지는 그 안에 모듈이 한 단계 더 있으므로 도트 역시 한 번 더 사용해야 합니다. 예를 들어, `matplotlib`의 모듈인 `pyplot`의 명령어 `a()`를 사용한다면 다음과 같이 코드를 작성합니다.

```
import matplotlib
matplotlib.pyplot.a()
```

`matplotlib.pyplot`은 `matplotlib`의 모듈인 `pyplot`에 접근한다는 의미이고, 여기에 `.a()`를 붙이면 `pyplot`에 있는 명령어 `a()`를 사용한다는 의미가 되죠. 도트는 이처럼 한 단계 아래로 접근할 수 있게 합니다.



▲ 그림 8-1 패키지 안의 명령어 사용하기

as로 명령어 줄여 쓰기

사용하고 싶은 명령어가 패키지나 모듈에 속해 있으면 명령어를 사용할 때마다 패키지 이름이나 모듈 이름을 넣어야 해서 번거롭습니다. 명령어가 길어지기도 하고요. 가령 `matplotlib`의 `pyplot` 모듈에 속한 명령어를 사용한다고 해 봅시다. 명령어 `b()`와 `c()`를 사용하려면 다음과 같이 작성해야겠죠.

```
import matplotlib
matplotlib.pyplot.b()
matplotlib.pyplot.c()
```

이런 경우 `as` 키워드를 사용해 줄여 표현하는 방법이 있습니다.

```
import matplotlib.pyplot as plt
```

`import matplotlib.pyplot`은 `matplotlib`의 `pyplot` 모듈을 포함한다는 의미입니다. 이때 코드 마지막에 `as plt`를 추가하면 앞에 있는 `matplotlib.pyplot`을 앞으로 `plt`로 부르겠다는 의미가 됩니다. 따라서 `pyplot`에 있는 명령어를 다음과 같이 줄여서 사용할 수 있습니다.

```
import matplotlib.pyplot as plt
plt.b()
plt.c()
```

다른 단어를 써도 있지만, 일반적으로 `matplotlib.pyplot`은 `plt`로 사용하니 이 책에서도 `plt`를 사용하겠습니다. 이 장에서는 `matplotlib`의 `pyplot`만 사용하므로 파일 시작 부분에서 `import matplotlib.pyplot as plt`를 한 번만 실행하면 그래프를 그리는 명령어를 계속 사용할 수 있습니다. 그럼 도구 사용법을 알았으니 본격적으로 그래프를 그려 봅시다.

> 졔졔쌤의 조언
> 지금까지는 주피터 노트북과 구글 코랩에서 코드를 실행했을 때 큰 차이가 없었습니다. 그런데 matplotlib의 일부 명령어는 조금 차이가 있습니다. 해당 부분에 설명을 추가해 두었으니 설명을 보고 충분히 따라할 수 있습니다.
>
> 또한, 코랩은 인터넷에 데이터 파일을 업로드해야 해서 데이터 파일 용량이 크면 업로드 시간이 오래 걸립니다. 하지만 주피터 노트북은 로컬에서 구동되기 때문에 내 PC에 파일이 있다면 업로드 없이 빠르게 데이터 파일을 읽어 들일 수 있습니다. 10장까지는 큰 데이터를 사용하지 않으므로 코랩을 사용해도 됩니다. 다만, 11장부터는 용량이 매우 큰 데이터를 사용하므로 주피터 노트북을 사용하길 추천합니다.

8.1.2 꺾은선 그래프 그리기

앞으로 여러 유형의 그래프를 그려 볼 텐데요. 그중에서 하나를 골라 제대로 이해한 후 다른 그래프로 넘어가겠습니다. 사용법이 비슷해서 하나를 제대로 알면 다른 그래프도 이해하기가 쉽습니다. 여기서는 꺾은선 그래프를 기준으로 설명합니다.

꺾은선 그래프는 `plot()` 명령어로 그립니다. 그래프를 그릴 데이터를 `plot()`의 소괄호 안에 넣고 실행하면 그래프가 그려집니다. `plot()`은 `pyplot` 모듈에 속한 명령어이므로 앞에서 배웠듯이 `plt.plot()`으로 사용합니다. 다음을 실행해 봅시다.

```
import matplotlib.pyplot as plt
plt.plot([1, 5, 7, 3, 7])
```

실행결과

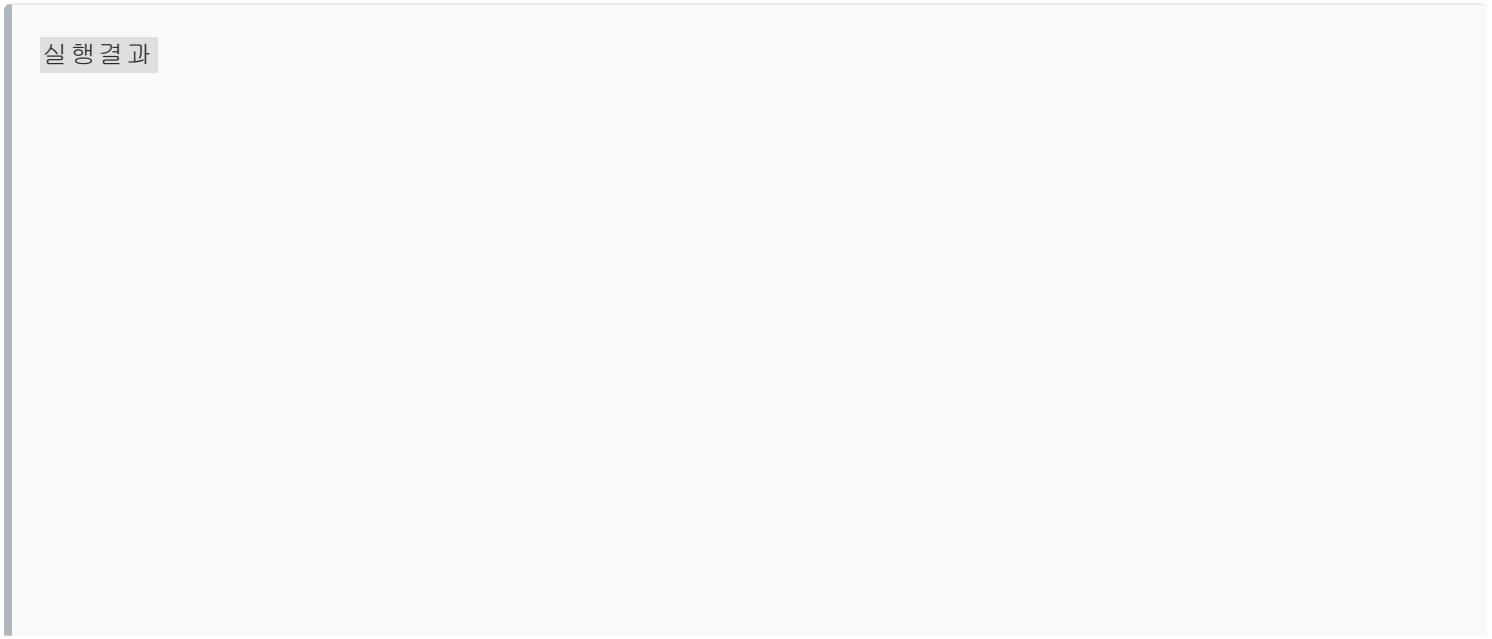
```
[<matplotlib.lines.Line2D at 0x234bfaffd60>]
```

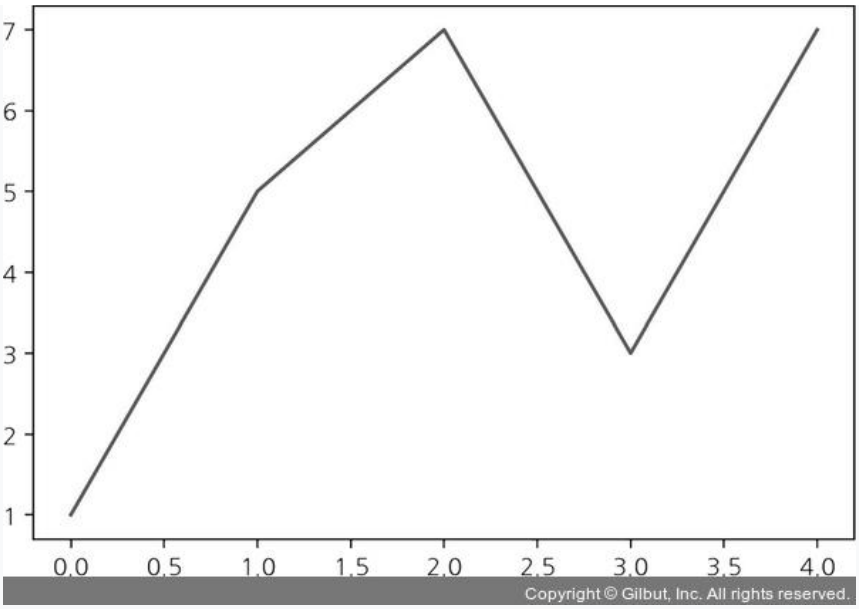
아무 일도 벌어지지 않네요. 정상적인 결과이니 당황하지 마세요. 그래프를 보여 주는 별도의 명령어가 필요해서 그렇습니다. `plt.show()`를 추가해 다시 실행해 봅시다.

TIP 앞의 코드를 실행했을 때 결과창에 텍스트와 그래프가 같이 표현될 수도 있습니다. 그래프를 볼 수 있더라도 그래프를 화면에 그리게 하는 `plt.show()`를 마지막에 넣는 것이 제대로 된 형식입니다.

```
import matplotlib.pyplot as plt
plt.plot([1, 5, 7, 3, 7])
plt.show()
```

실행결과




7
6
5
4
3
2
1
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Copyright © Gilbut, Inc. All rights reserved.

드디어 그래프가 그려집니다. 코드와 결과를 살펴봅시다. `plot()`에 넣은 리스트 `[1, 5, 7, 3, 7]`은 x축일까요? y축일까요? 그래프를 보면 y축으로 보이죠? 이처럼 `plt.plot()`은 데이터가 단일 값이면 입력된 값을 y축으로 보고 x축을 기본으로 0부터 설정됩니다. 만약 x축에도 특정 값을 넣고 싶다면 x축 데이터도 추가하면 됩니다. 2개의 데이터를 넣으면 앞은 x축, 뒤는 y축으로 인식해 그래프를 그립니다.

형식

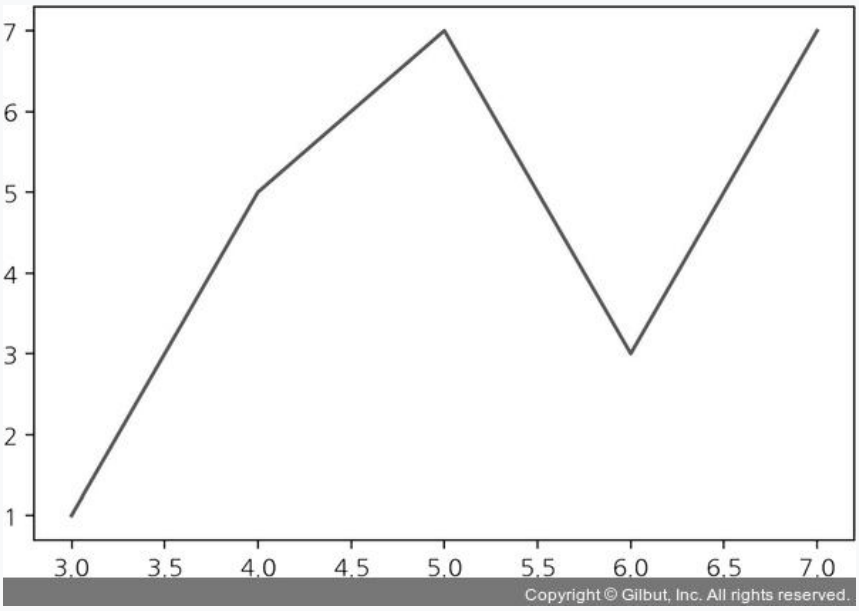
```
plt.plot(x축, y축)
```

그래프를 그리려면 앞에서처럼 여러 개의 데이터 즉, 데이터 묶음이 필요합니다. 이 책에서는 리스트로 그래프를 그리겠습니다.

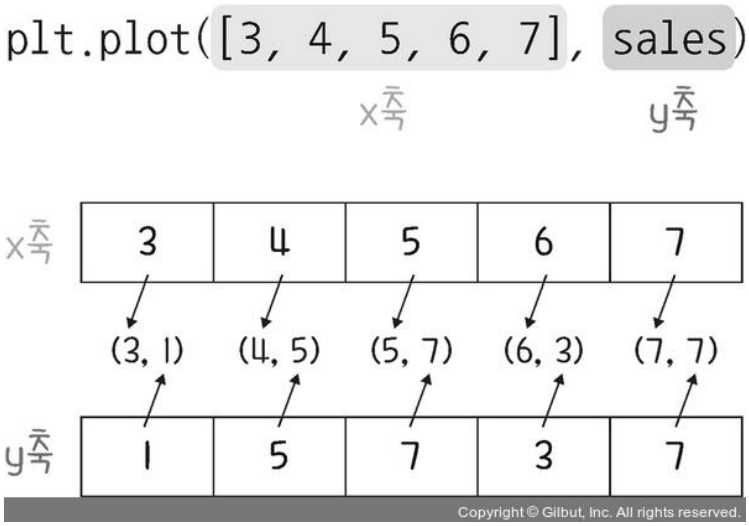
앞에서는 `plt.plot()`에 리스트를 직접 넣었지만, 리스트를 변수에 저장하고 변수를 넣는 것이 일반적입니다. `[1, 5, 7, 3, 7]`을 3월부터 7월까지 5개월 간의 매출이라고 가정해 봅시다. 그럼 3월부터 7월을 리스트 `[3, 4, 5, 6, 7]`로 만들고, 이를 변수 `month`에 저장합니다. `[1, 5, 7, 3, 7]`도 변수 `sales`에 저장합니다. `month`와 `sales`를 `plt.plot()`에 넣으면 각각 x축과 y축을 나타내는 리스트가 되겠죠? 다음 코드를 실행해 봅시다.

```
month = [3, 4, 5, 6, 7]
sales = [1, 5, 7, 3, 7]
plt.plot(month, sales)
plt.show()
```

실행결과



`plt.plot()`에 `month`를 넣으니 이를 x축으로 인식해 x축이 3부터 7 사이의 값으로 바뀌었습니다. y축은 여전히 `sales`의 값으로 대응되고요.



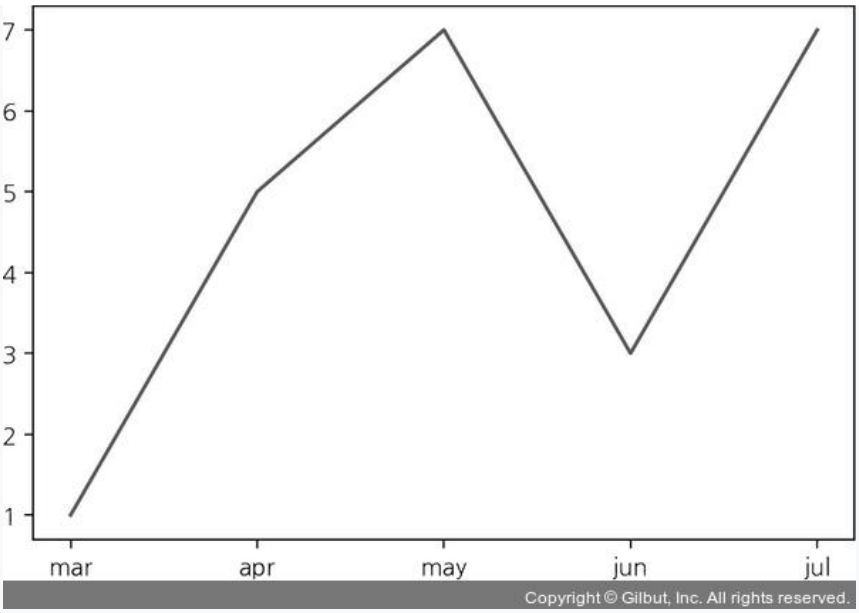
▲ 그림 8-2 꺾은선 그래프의 x축과 y축

TIP 그래프를 그릴 때 눈금 간격은 지정하지 않으면 간격이 자동으로 설정됩니다. 그래서 앞의 결과처럼 3.5, 4.5와 같은 소수점이 있는 실수가 나옵니다. 자세한 내용은 뒤에서 배우겠습니다.

이번에는 의미를 조금 더 알기 쉽게 x축의 '월'을 문자로 바꿔 봅시다. 3월부터 7월을 영어 약자로 변경해 리스트에 넣습니다.

```
month = ['mar', 'apr', 'may', 'jun', 'jul']
sales = [1, 5, 7, 3, 7]
plt.plot(month, sales)
plt.show()
```

실행결과


7
6
5
4
3
2
1
mar
apr
may
jun
jul
Copyright © Gilbut, Inc. All rights reserved.

`month`를 `[3, 4, 5, 6, 7]`에서 `['mar', 'apr', 'may', 'jun', 'jul']`로 변경하니 x축이 `['mar', 'apr', 'may', 'jun', 'jul']`로 변경됐습니다.

NOTE 자주 하는 실수 - 데이터 개수 맞추기

`plt.plot()`에서 x축과 y축에 데이터를 넣을 때 오류가 자주 발생하는 부분이 있습니다. 다음 코드를 실행해 보고 왜 오류가 났는지 생각해 봅시다.

```
sales = [1, 5, 7, 3, 7]
plt.plot([3, 4, 5, 6, 7, 8], sales)
plt.show()
```

```
실행결과
----> 2 plt.plot([3, 4, 5, 6, 7, 8], sales)
ValueError: x and y must have same first dimension, but have shapes (6,) and (5,)
```

오류 설명을 보면 'x와 y는 첫 번째 차원이 반드시 같아야 하는데, 이 코드는 (6,)과 (5,) 형태여서 ValueError가 발생한다'고 나옵니다. 여기서 형태라고 하는 (6,)과 (5,)는 리스트의 요소 개수를 의미합니다. x축은 `[3, 4, 5, 6, 7, 8]`로 6개인데, y축은 `[1, 5, 7, 3, 7]`로 5개입니다. 이렇게 x축과 y축의 데이터 개수가 같지 않으면 그래프를 그릴 수 없습니다. 그래프에 값을 넣을 때는 데이터 개수가 같은지 항상 확인하세요!

8.1.3 그래프 꾸미기

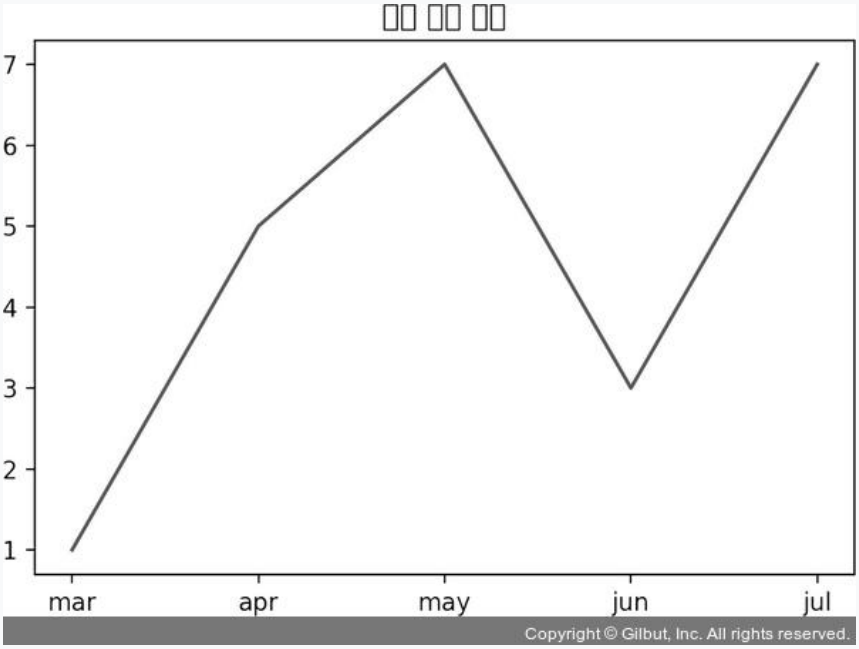
이번에는 그래프를 보기 좋게 꾸며 보겠습니다.

제목 넣기

먼저 그래프에 제목을 붙여 봅시다. 제목은 영어로 title이죠. 그래서 그래프에 제목을 붙이는 명령어도 `plt.title()`입니다. 원하는 그래프 제목을 `plt.title()`의 소괄호에 넣고 실행해 봅시다.

```
sales = [1, 5, 7, 3, 7]
month = ['mar', 'apr', 'may', 'jun', 'jul']
plt.title('월별 판매 실적') # 그래프 제목 넣기
plt.plot(month, sales)
plt.show()
```

실행결과



영어로 된 x축은 문제없이 잘 출력되지만, 한글로 입력한 제목은 제대로 표시되지 않습니다. 이는 `matplotlib`이 기본으로 한글을 지원하지 않아서 그렇습니다. 한글이 나오게 하려면 다음 형식으로 한글 글꼴을 지정하는 명령어를 추가해야 합니다.

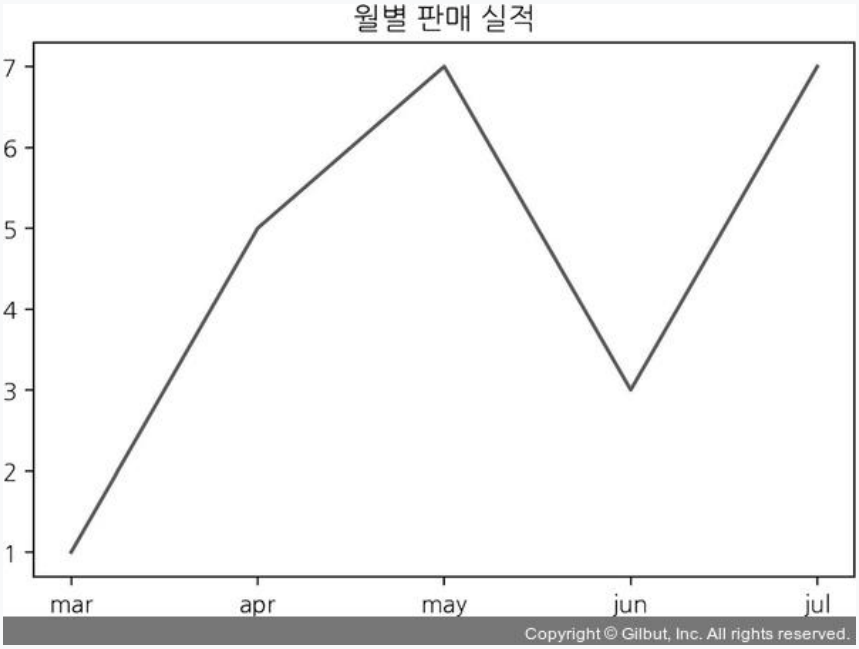
형식

```
plt.rc('font', family='글꼴 이름')
```

다음처럼 코드를 추가해 봅시다.

```
sales = [1, 5, 7, 3, 7]
month = ['mar', 'apr', 'may', 'jun', 'jul']
plt.rc('font', family='Malgun Gothic') # 글꼴 설정, macOS에서는 AppleGothic
plt.title('월별 판매 실적')            # 그래프 제목 넣기
plt.plot(month, sales)
plt.show()
```

실행결과



보통 그래프 제목이 있고 그 아래 그래프가 나오죠. 그래서 `plt.title()`이 `plt.plot()`보다 위에 있습니다. 그리고 그래프 제목을 한글로 출력하려면 그래프 제목을 출력하기 전에 한글 글꼴을 설정해야 하므로 `plt.rc()`를 `plt.title()`보다 위에 작성합니다. 글꼴은 Windows일 경우 Malgun Gothic을, macOS일 경우 AppleGothic을 넣으면 됩니다. Malgun Gothic은 중간에 공백이 있으니 유의하세요!

NOTE 코랩에서 한글 글꼴 설치하기

주피터 노트북은 로컬 PC에서 글꼴을 가져오기 때문에 실습하는 컴퓨터에 지정한 글꼴이 있으면 명령어만으로 한글 글꼴을 지정할 수 있습니다. 하지만 코랩은 클라우드에서 작동하기 때문에 한글 글꼴을 설정하려면 글꼴을 따로 설치해야 합니다.

여기서는 무료 글꼴인 나눔 글꼴을 설치합니다. 다음 코드를 작성 중인 코랩 파일의 첫 번째 셀에서 실행합니다.

```
!sudo apt-get install -y fonts-nanum # 코랩의 첫 번째 코드 셀에서 실행
!sudo fc-cache -fv
!rm ~/.cache/matplotlib -rf
```

설치가 끝나면 코랩 메뉴에서 런타임 → 런타임 다시 시작을 선택하고 팝업창이 뜨면 [예]를 클릭합니다.

주피터 노트북과 동일한 방법으로 글꼴을 지정하고 코드를 실행합니다.

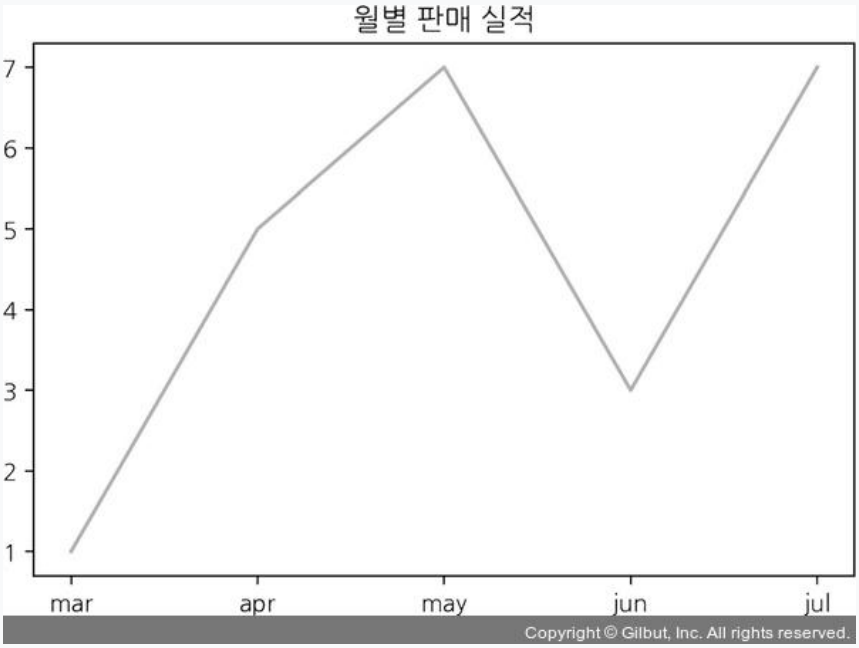
```
plt.rc('font', family='NanumGothic')
```

그래프 색 바꾸기

이번에는 그래프의 색을 바꿔 보겠습니다. 색을 바꾸려면 `plt.plot()`에 `color` 옵션을 추가합니다. `color` 옵션의 값을 yellow를 의미하는 `y`로 설정하고 실행해 봅시다.

```
plt.rc('font', family='Malgun Gothic')
plt.title('월별 판매 실적')
plt.plot(month, sales, color='y') # 그래프 색 바꾸기
plt.show()
```

실행결과



그래프가 노란색으로 바뀝니다. 다른 색을 넣고 실행해 보면 그래프의 색이 변하는 걸 볼 수 있습니다. 색상 코드는 https://matplotlib.org/examples/color/named_colors.html을 참고하세요.

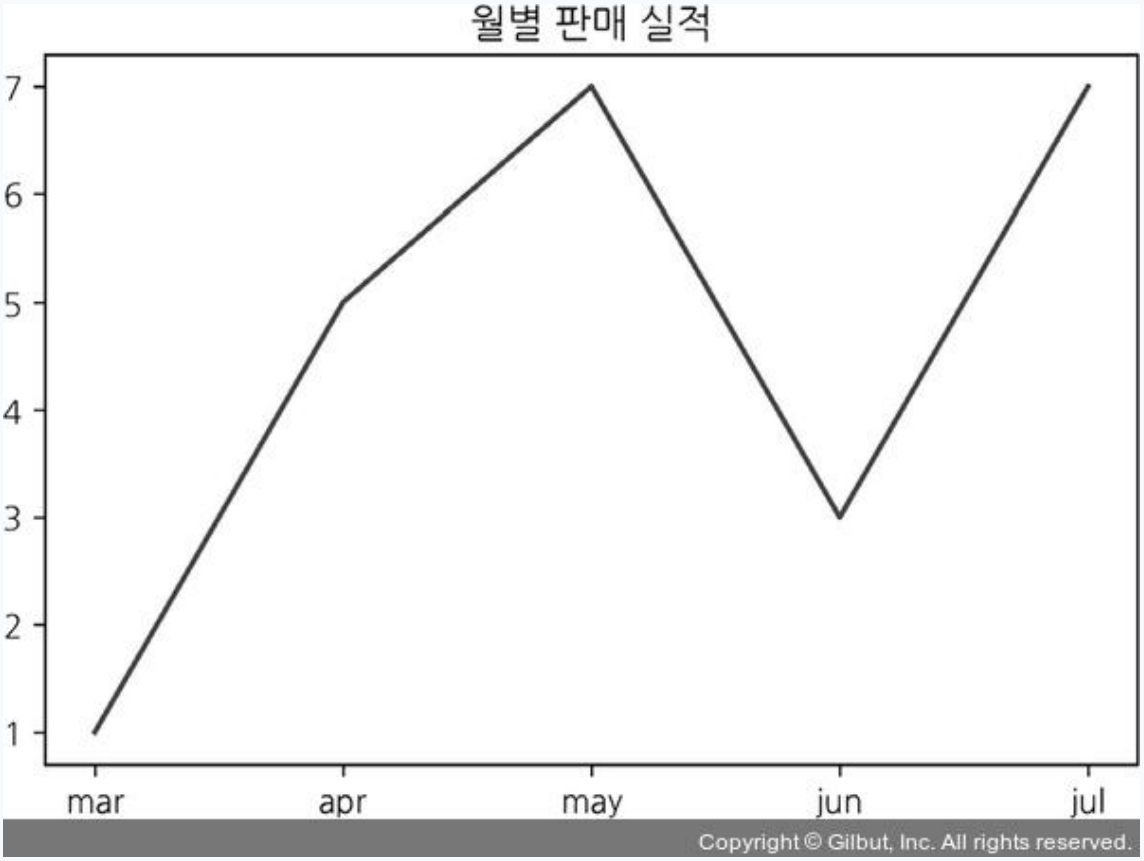
크기 바꾸기

화면은 큰데 결과 그래프가 조금 작아 보입니다. 이럴 때는 `plt.figure()`를 사용해 그래프 크기를 조절합니다. 크기를 조절할 때는 `figsize`와 `dpi`라는 2가지 옵션을 줄 수 있습니다. `figsize`는 `figsize=(가로, 세로)` 형식으로, 그래프의 가로와 세로 크기를 인치 단위로 직접 지정합니다. `dpi`는 해상도로 크기를 조절합니다. dpi는 dot per inch의 약어로, 해상도를 나타내는 단위입니다. 1제곱인치 안에 점(dot), 즉 픽셀이 몇 개 있는지를 의미합니다.

여기서는 `dpi` 옵션으로 크기를 조절합니다. 값을 150으로 설정하고 실행합니다.

```
plt.figure(dpi=150) # 그래프 크기 바꾸기
# plt.figure(figsize=(8, 4))
plt.rc('font', family='Malgun Gothic')
plt.title('월별 판매 실적')
plt.plot(month, sales, color='b')
plt.show()
```

실행결과

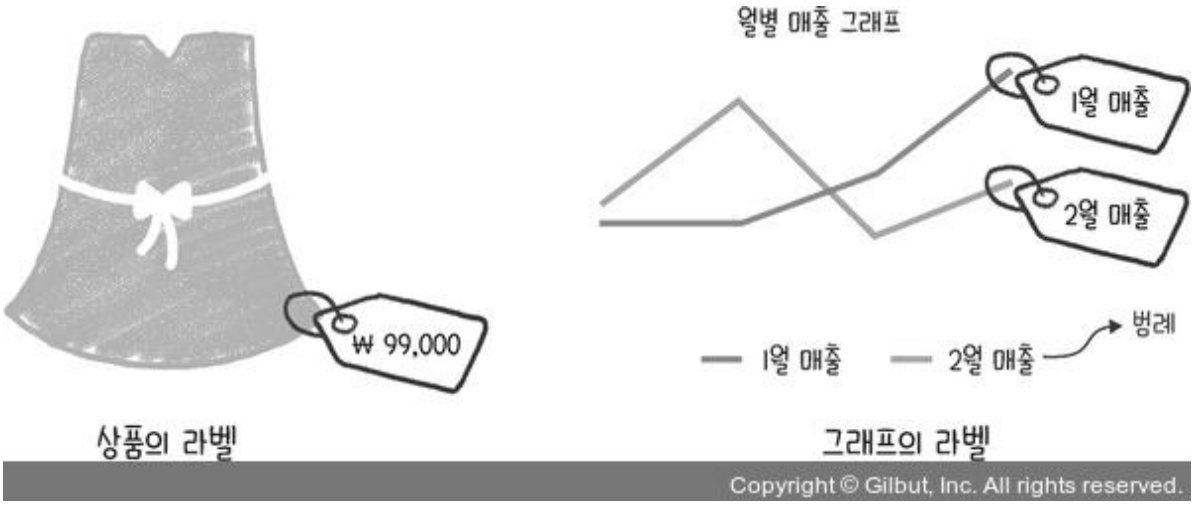


그래프 크기가 커지면서 글자가 조금 더 잘 보입니다. 크기를 더 키우고 싶을 때는 더 큰 숫자를 넣으면 됩니다. `figsize`로도 실습해 보세요.

범례 넣기

이번에는 범례를 추가해 보겠습니다. 그래프에서 점, 선, 막대의 색이나 모양에 따른 의미를 설명하는 부분을 범례(legend)라고 합니다. 보통 그래프의 한쪽에 글상자 형태로 표시됩니다. 그래프에 범례를 표시하려면 `plt.plot()`에 `label` 옵션으로 라벨을 붙이면 됩니다.

흔히 종이나 천에 상표나 제품명 등을 인쇄해 상품에 붙여 놓은 조각을 라벨(또는 레이블, label)이라고 합니다. 상품에 라벨이 달려 있듯이 그래프의 구성 요소가 의미하는 바를 나타내고 싶을 때 라벨을 사용합니다.

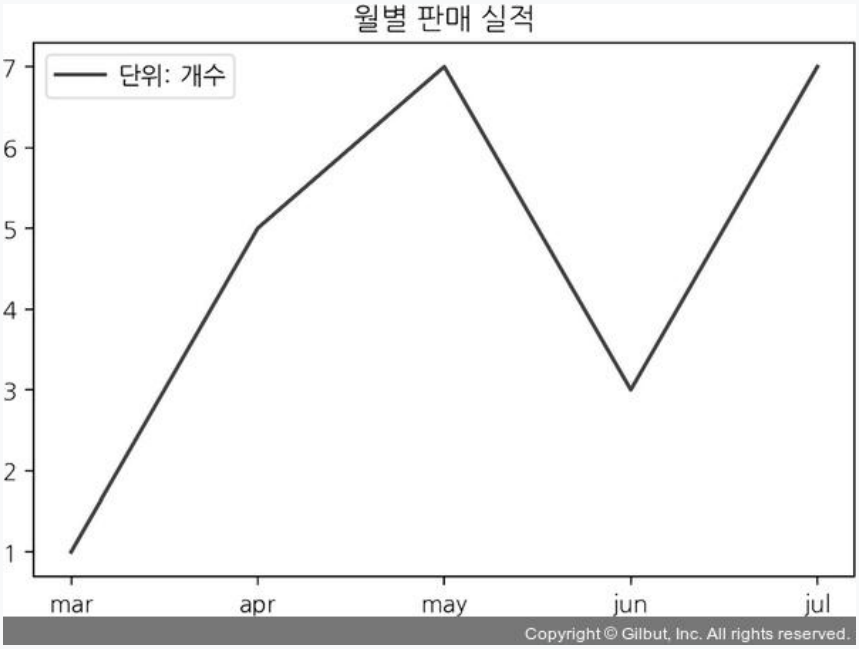


▲ 그림 8-3 라벨

`plt.plot()`에 `label` 옵션을 추가하면 그림처럼 그래프 선에 라벨이 달립니다. 다만, 그래프에서는 라벨이 상품 라벨과 같은 형태로 나타나는 것이 아니라 범례의 형태로 나타납니다. `matplotlib`으로 그래프에 라벨을 붙이려면 2가지가 필요합니다. 먼저 `plt.plot()`에 `label` 옵션으로 라벨 내용을 설정하고, `plt.legend()`로 그래프에 범례를 표시합니다. 일단 코드를 실행해 봅시다.

```
plt.rc('font', family='Malgun Gothic')
plt.title('월별 판매 실적')
plt.plot(month, sales, color='b', label='단위: 개수') # 범례 넣기
plt.legend()
plt.show()
```

실행결과

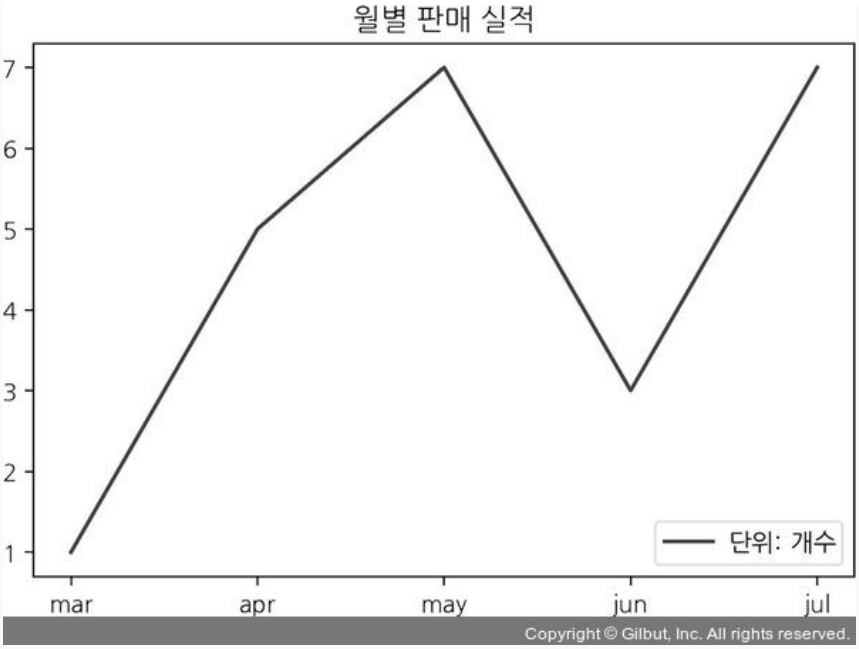


`label='단위: 개수'`로 그래프의 라벨을 지정하고 `plt.legend()`로 범례를 표시합니다. `plt.legend()`를 추가하지 않으면 라벨 옵션을 줘도 범례가 표시되지 않습니다.

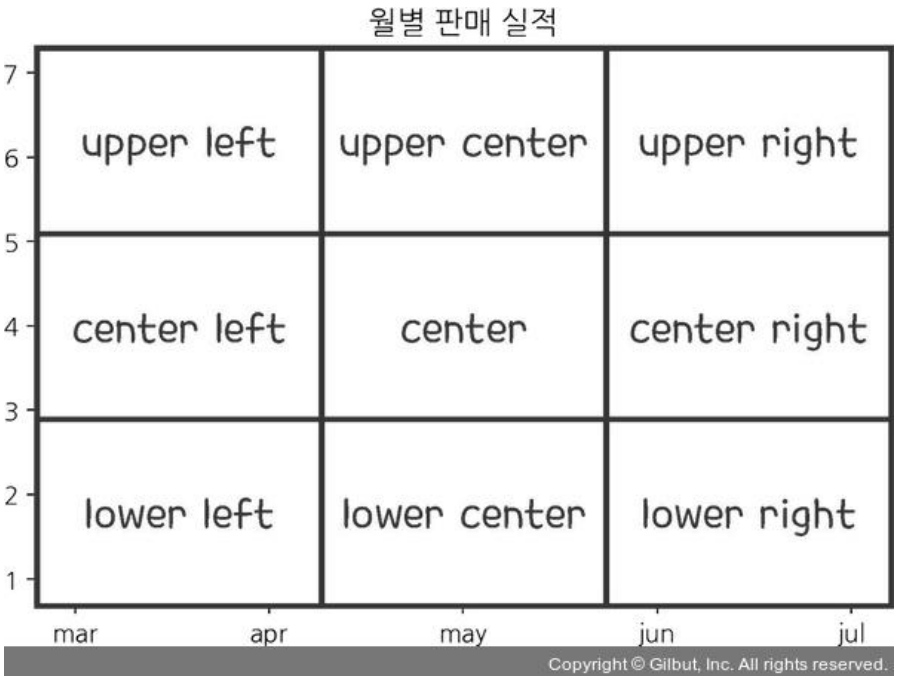
`plt.legend()`에 `loc` 옵션을 추가하면 범례 위치를 바꿀 수 있습니다.

```
plt.rc('font', family='Malgun Gothic')
plt.title('월별 판매 실적')
plt.plot(month, sales, color='b', label='단위: 개수')
plt.legend(loc='lower right')
plt.show()
```

실행결과


월별 판매 실적
7
6
5
4
3
2
1
mar
apr
may
jun
jul
단위: 개수
Copyright © Gilbut, Inc. All rights reserved.

범례가 오른쪽 아래로 옮겨집니다. 범례 위치는 다음과 같이 그래프를 9등분해 지정하고, `best`로 지정하면 그래프의 생김새에 따라 자동으로 최적이라 판단되는 위치에 범례를 표시합니다.



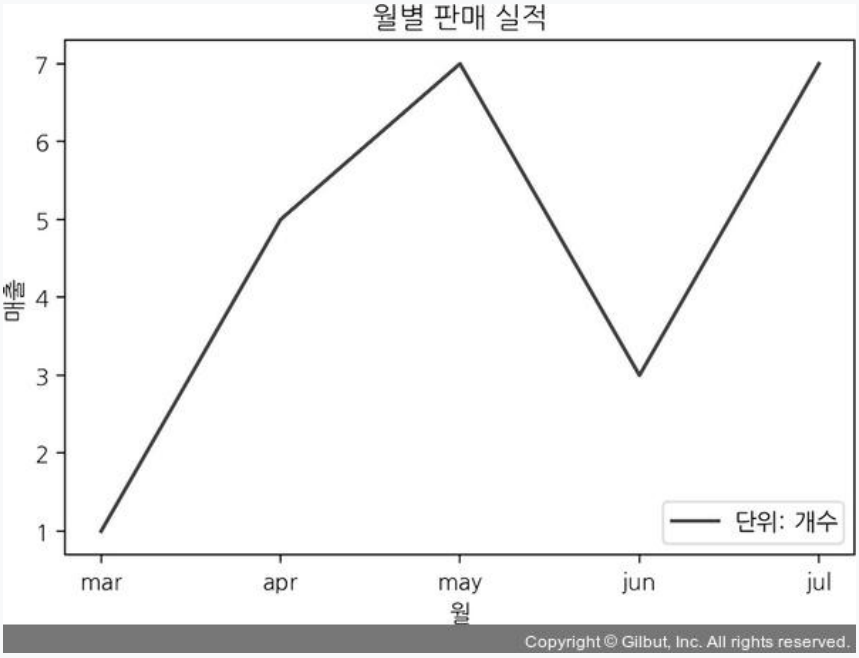
▲ 그림 8-4 옵션 값에 따른 범례 위치

축 이름 넣기

그래프에 x축과 y축의 이름이 없으니 의미를 한 번에 알기 어렵네요. 이번에는 x축과 y축에 이름을 붙여 봅시다. x축에 이름을 붙이는 명령어는 `plt.xlabel()`, y축에 이름 붙이는 명령어는 `plt.ylabel()`입니다. 축 이름은 소괄호 안에 넣습니다.

```
plt.rc('font', family='Malgun Gothic')
plt.title('월별 판매 실적')
plt.plot(month, sales, color='b', label='단위: 개수')
plt.legend(loc='lower right')
plt.xlabel('월')
plt.ylabel('매출')
plt.show()
```

실행결과



x축과 y축에 축 이름이 표시됩니다. `plt.xlabel()`과 `plt.ylabel()`은 `label` 옵션과 비슷해 보이지만, 명령어에 추가되는 옵션이 아니라 별도의 명령어라는 점을 유의하세요!

지금까지 꺾은선 그래프를 그리고 여러 옵션으로 그래프를 꾸며 봤습니다. 앞에서 봤듯이 그래프를 그릴 때는 일정한 형식이 있습니다. 가장 먼저 라이브러리를 추가하고 `plt.title()`로 제목을 붙이고 `plt.plot()`으로 그래프를 그리는 것이죠. 만든 그래프를 보여 주지 않으면 소용이 없으니 마지막에 그래프를 보여 주는 명령어 `plt.show()`도 필수입니다. 정리하면 다음과 같습니다.

```
import matplotlib.pyplot as plt    # plt로 명령어 줄여 쓰기

plt.rc('font', family='글꼴 이름') # 한글 글꼴 지정하기
plt.title('그래프 제목')           # 그래프 제목 넣기
plt.plot(x축 데이터, y축 데이터)   # 꺾은선 그래프 그리기
plt.show()                         # 그래프 보여 주기
```

여기에 해상도, 범례, 색상 등 그래프를 꾸미는 명령어를 선택적으로 추가할 수 있습니다.

1분 퀴즈

1 다음 코드를 보고 틀린 설명을 고르세요.

```
import matplotlib.pyplot as plt

mon = [1, 2, 3, 4, 5, 6, 7, 8, 9]
data = [92, 63, 8, 27, 39, 84, 76, 18, 46]

plt.figure(dpi=200)
plt.rc('font', family='Malgun Gothic')
plt.plot(mon, data, color='r', label='지출')
plt.legend(loc='upper left')
plt.show()
```

① x축의 범위는 1부터 9다.

② 범례는 왼쪽 위에 표시된다.

③ '지출'이라는 그래프 제목이 나타난다.

④ `import matplotlib.pyplot as plt`는 `matplotlib.pyplot`을 줄여 `plt`로 사용할 수 있게 한다.

⑤ `plt.rc('font', family='Malgun Gothic')`은 한글 글꼴을 설정하는 코드로, 이 코드가 없다면 그래프에서 한글이 표시되지 않는다.

정답 및 해설

8.2

다양한 유형의 그래프 그리기

꺾은선 그래프 외에도 다양한 그래프를 그릴 수 있습니다. 기본 구조는 같고, 그래프를 그리는 명령어 `plt.plot()`만 바꿀 뿐이죠. 그럼 하나씩 살펴봅시다.

8.2.1 막대그래프 그리기

막대그래프는 꺾은선 그래프만큼 자주 쓰입니다. 응용할 수 있는 부분도 무궁무진하죠. 막대그래프는 `plt.bar()`로 그립니다. 꺾은선 그래프를 그린 것처럼 다음 코드로 막대그래프를 그려 봅시다.

```
import matplotlib.pyplot as plt

sales = [1, 5, 7, 3, 7]
plt.bar(sales)
plt.show()
```

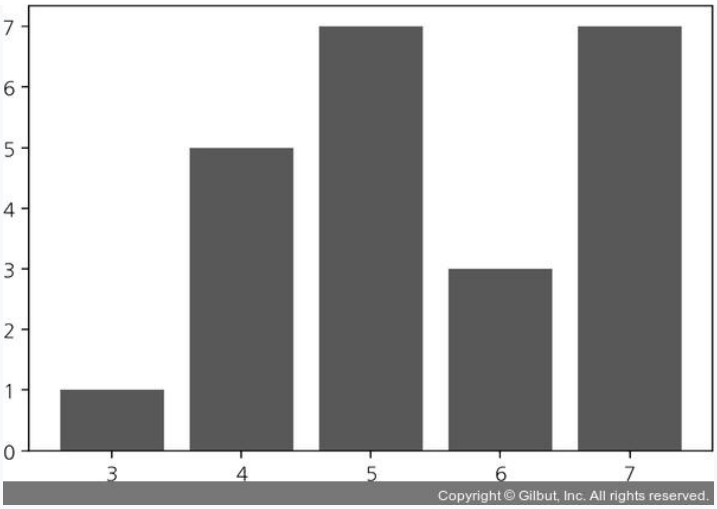
```
실행결과
----> 4 plt.bar(sales)
TypeError: bar() missing 1 required positional argument: 'height'
```

오류가 뜨지만 당황하지 말고 오류 내용을 잘 살펴봅시다. 마지막 줄을 보니 `TypeError`가 발생했고, '`bar()`에 요구되는 위치 요소인 높이(height)가 누락됨'이라고 합니다. 즉, `bar()`에는 높이도 넣어 줘야 하는데, 빠뜨렸다는 뜻입니다.

`plt.plot()`은 데이터를 하나만 넣어도 됐지만, `plt.bar()`는 2개의 데이터(x축, y축)를 모두 넣어야 그래프를 그립니다. 그럼 데이터를 추가해 다시 실행해 봅시다.

```
sales = [1, 5, 7, 3, 7]
month = [3, 4, 5, 6, 7]
plt.bar(month, sales)
plt.show()
```

```
실행결과
```



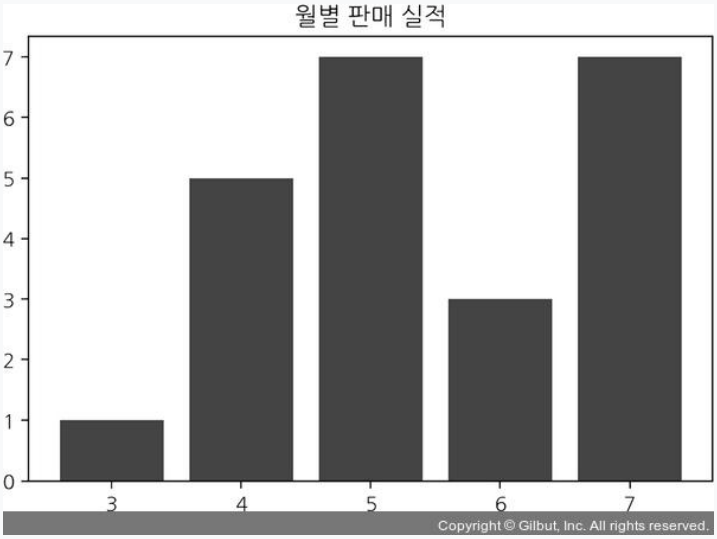
막대그래프가 잘 그려집니다. x축에 `month`의 값이 차례대로 들어가고, y축에도 `sales`의 값이 차례대로 들어갑니다.

제목 넣고 그래프 색 바꾸기

이번에는 그래프에 제목도 넣고, 막대도 파란색으로 바꿔 봅시다. 제목은 `plt.title()`로 추가하면 됩니다. 제목은 한글이니 글꼴도 설정해야죠. 파란색은 `color` 옵션을 추가해 `'blue'` 또는 `'b'`로 설정하면 됩니다.

```
sales = [1, 5, 7, 3, 7]
month = [3, 4, 5, 6, 7]
plt.rc('font', family='Malgun Gothic')
plt.title('월별 판매 실적')
plt.bar(month, sales, color='b')
plt.show()
```

실행결과



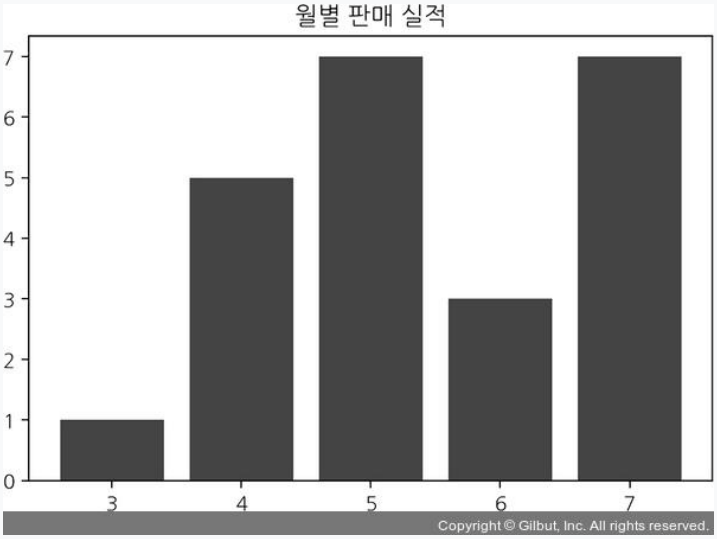
꺾은선 그래프에서 배운 형식을 그대로 적용해서 그래프 제목과 색을 출력했습니다. 범례를 적용하는 것 역시 꺾은선 그래프와 같으니 직접 수정해 보세요!

x축에 range()로 연속 숫자 넣기

반복문에서 연속되는 숫자 범위를 표현할 때 `range()`라는 명령어를 배웠습니다. 리스트 `[3, 4, 5, 6, 7]`도 3부터 7까지 연속된 숫자이므로 `range()`를 사용해 `range(3, 8)`로 표현할 수 있습니다. x축에 `[3, 4, 5, 6, 7]` 대신 `range(3, 8)`을 넣고 실행해 봅시다.

```
sales = [1, 5, 7, 3, 7]
plt.rc('font', family='Malgun Gothic')
plt.title('월별 판매 실적')
plt.bar(range(3, 8), sales, color='b')
plt.show()
```

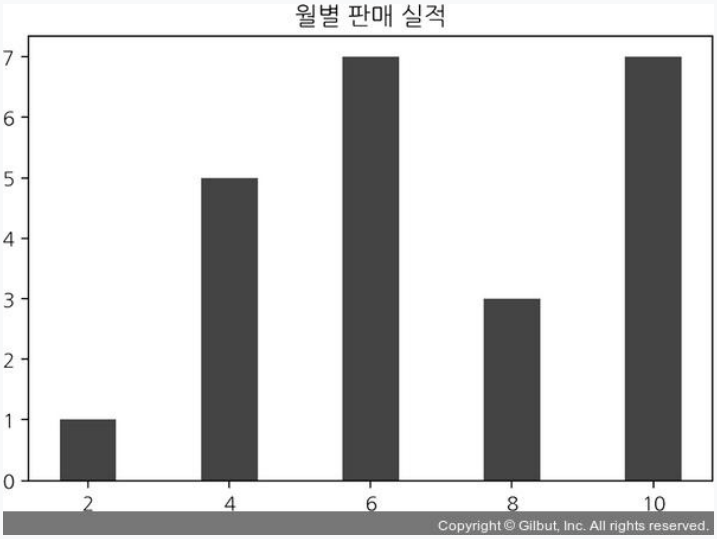
실행결과



x축 데이터가 `[3, 4, 5, 6, 7]`일 때와 똑같은 그래프가 그려집니다. `range()`의 숫자를 변경하면 다른 달의 그래프로도 그릴 수 있겠죠? 예를 들어, `range(1, 6)`을 넣으면 1월부터 5월까지, `range(2, 11, 2)`를 넣으면 2부터 시작해 2씩 건너뛰므로 짝수 월로 x축을 설정할 수 있습니다.

```
sales = [1, 5, 7, 3, 7]
plt.rc('font', family='Malgun Gothic')
plt.title('월별 판매 실적')
plt.bar(range(2, 11, 2), sales, color='b')
plt.show()
```

실행결과



이처럼 데이터가 연속된 숫자일 때는 리스트 대신 `range()` 명령어로 데이터를 넣을 수도 있습니다.

NOTE 자주 하는 실수 - 데이터 개수 맞추기

`plt.bar()`에서도 x축 데이터의 개수와 y축 데이터의 개수가 같지 않으면 오류가 발생합니다.

```
sales = [1, 5, 7, 3, 7]
plt.rc('font', family='Malgun Gothic')
plt.title('월별 판매 실적')
plt.bar(range(6), sales, color='b')
plt.show()
```

```
실행결과
----> 4 plt.bar(range(6), sales, color='b')
ValueError: shape mismatch: objects cannot be broadcast to a single shape
```

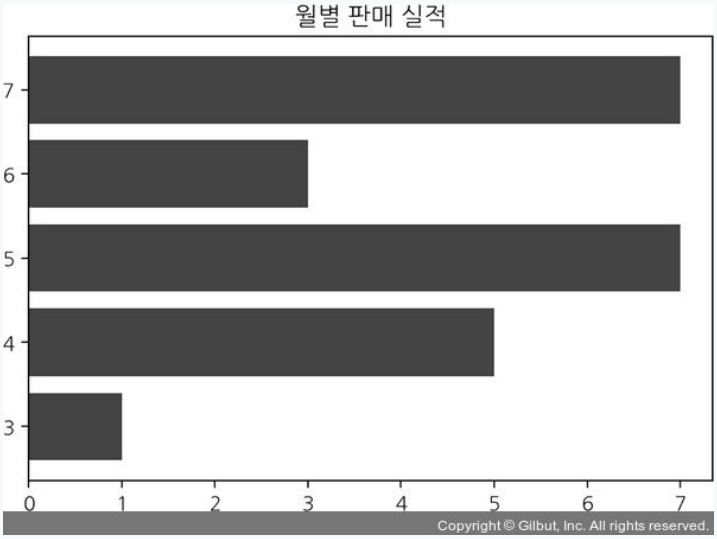
오류 내용이 길어도 화살표가 가리키는 부분과 마지막 내용만 봐도 오류를 이해할 수 있으니 두려워하지 마세요. 화살표가 `plt.bar(range(6), sales, color='b')`를 가리키죠? 이 부분에서 오류가 발생했음을 알 수 있습니다. 마지막 줄을 보면 '형태 불일치: 대상들이 하나의 형태로 뿌려질 수 없다'고 나옵니다. 복잡해 보이지만, 이는 x축 데이터와 y축 데이터의 형태가 일치하지 않아서 오류가 발생한다는 뜻입니다. `range(6)`은 0부터 5까지를 의미하므로 값이 총 6개지만, `sales`에는 5개만 있습니다. 그래프를 그릴 때는 x축 데이터와 y축 데이터의 개수가 같아야 한다는 점을 꼭 기억해 두세요.

막대 방향 전환하기

막대가 세로로 선 형태(수직)가 아니라 가로로 누운 형태(수평)로도 그래프를 그릴 수 있습니다. `plt.bar()`에 한 글자를 추가한 `plt.barh()`를 사용하면 됩니다. 여기에서 h는 horizontal을 의미합니다.

```
sales = [1, 5, 7, 3, 7]
plt.rc('font', family='Malgun Gothic')
plt.title('월별 판매 실적')
plt.barh(range(3, 8), sales, color='b')
plt.show()
```

실행결과



`plt.barh()`는 `plt.bar()`와 사용법이 같고, 기준점만 x축이 아닌 y축으로 바뀝니다. 따라서 `range(3, 8)`이 y축이 되고, `sales`가 x축이 됩니다.

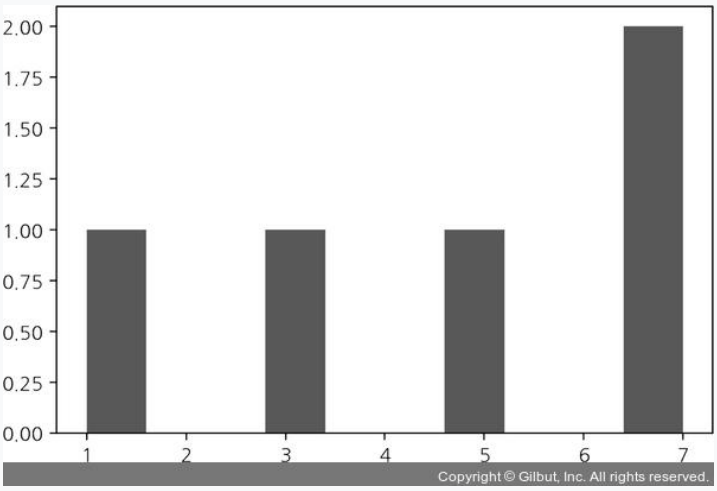
꺾은선 그래프를 배울 때 적용한 방식과 같으니 막대그래프 그리기도 어렵지 않죠? 그래프 그리기 명령어만 `plt.plot()`에서 `plt.bar()`나 `plt.barh()`로 바꾸면 됩니다. 단, `plt.bar()`와 `plt.barh()`에는 반드시 x축과 y축 데이터를 모두 넣어야 합니다.

8.2.2 히스토그램 그리기

이번에는 히스토그램을 그려 보겠습니다. 히스토그램은 막대 모양의 도형으로 나타내는 그래프로, x축에는 데이터를, y축에는 데이터의 개수(빈도)를 넣어 분포를 표현합니다. 히스토그램을 그리는 명령어는 `plt.hist()`입니다. 다음 코드로 히스토그램을 그려 봅시다.

```
sales = [1, 5, 7, 3, 7]
plt.hist(sales)
plt.show()
```

실 행 결 과



`plt.hist()`는 데이터를 넣으면 데이터의 최솟값과 최댓값을 골라내서 최솟값을 x축의 가장 왼쪽에, 최댓값을 x축의 가장 오른쪽에 놓고 범위를 지정합니다. `[1, 5, 7, 3, 7]`에서 최솟값은 1, 최댓값은 7이죠. 그리고 1이 1개, 3이 1개, 5가 1개, 7이 2개입니다. 따라서 x축은 `[1, 3, 5, 7]`이 되고, y축은 `[1, 1, 1, 2]`가 됩니다. 이처럼 `plt.hist()`는 리스트를 하나만 넣어도 자동으로 리스트의 값이 다른 요소를 x축으로, 리스트의 각 요소 개수를 y축으로 설정합니다.

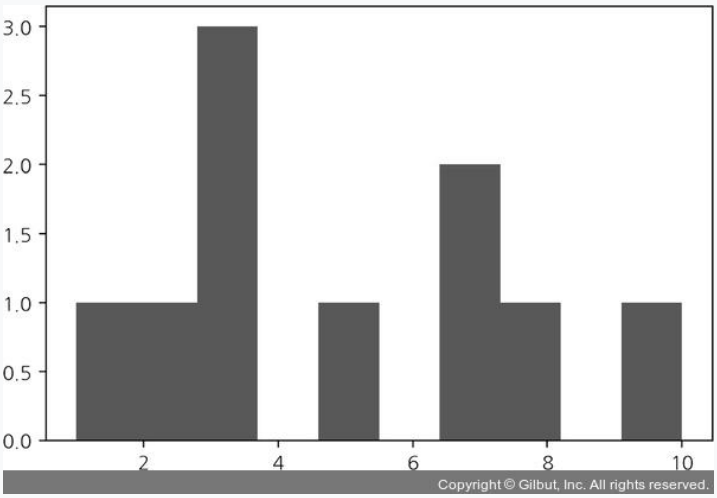
데이터가 적어서 막대그래프와 비슷해 보이네요. `random` 라이브러리를 사용해 무작위 정수 10개를 만들고, 이 데이터로 히스토그램을 그려 봅시다. 무작위 정수 10개를 저장하는 리스트명은 `a`라고 하겠습니다.

```
import random

a = []
for i in range(10):
    a.append(random.randint(1, 10))
print(a)
plt.hist(a)
plt.show()
```

실행결과

```
[7, 5, 3, 3, 1, 3, 2, 10, 7, 8]
```



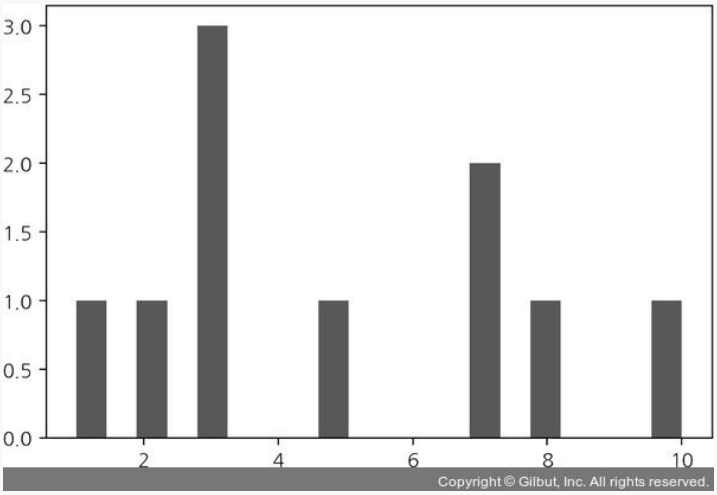
책과 결과가 다르다고 놀라지 마세요. 무작위 정수이므로 그래프 모양이 다를 수밖에 없습니다. 코드를 봅시다. 반복문을 활용해 `a` 리스트에 1부터 10 사이의 무작위 정수 10개를 추가했습니다. 반복문이 끝나면 `a`에 들어 있는 데이터를 출력해 확인하고 바로 `plt.hist(a)`로 히스토그램을 그렸습니다.

구간 조정하기

히스토그램을 그릴 때 구간을 조정할 수 있습니다. 구간은 막대의 너비라고 보면 됩니다. 옵션을 `bins`로 주는데, 이때 bin이 구간을 뜻합니다. 앞의 코드에서 얻은 무작위 값을 넣어 구간이 20인 히스토그램을 그려 보겠습니다.

```
a = [7, 5, 3, 3, 1, 3, 2, 10, 7, 8]
plt.hist(a, bins=20)
plt.show()
```

실 행 결 과



`bins`로 구간 개수를 설정하면 데이터의 x축 범위를 산출한 후 설정한 구간 개수로 나눕니다. 여기서는 1부터 10까지를 20개 구간으로 나누므로 처음 히스토그램보다 막대가 얇아졌습니다.

`bins`는 히스토그램에만 있는 옵션으로 다른 그래프 명령어에서는 사용할 수 없습니다. 그리고 `plt.hist()`는 리스트가 하나만 필요합니다. 따라서 x축과 y축 데이터를 모두 넣으면 오류가 발생합니다.

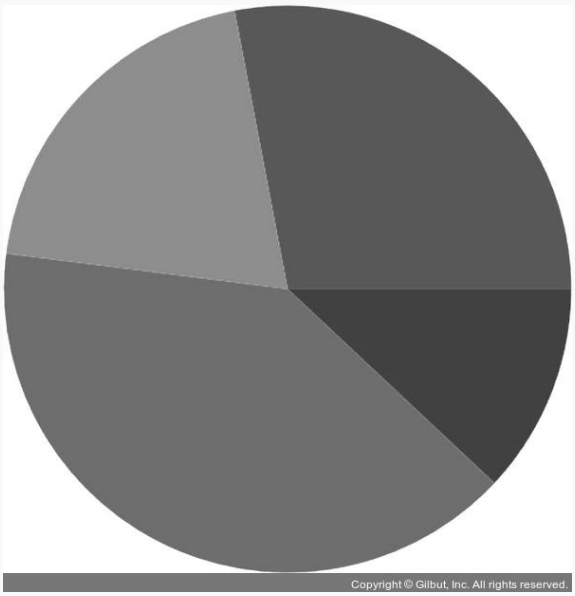
8.2.3 원그래프 그리기

원그래프도 그려 봅시다. 원그래프는 파이를 닮았다고 해서 '파이차트'라고도 합니다. 원그래프를 그리는 명령어는 `plt.pie()`이며, 히스토그램처럼 리스트 하나를 넣으면 됩니다.

혈액형별 사람 수를 원그래프로 그려 보겠습니다. 혈액형을 뜻하는 blood type을 줄인 `b_type`으로 리스트명을 짓고 `[25, 19, 37, 11]`을 저장합니다. 이를 각각 A형, B형, O형, AB형인 사람 수라고 가정합시다.

```
b_type = [25, 19, 37, 11]
plt.pie(b_type)
plt.show()
```

실행결과

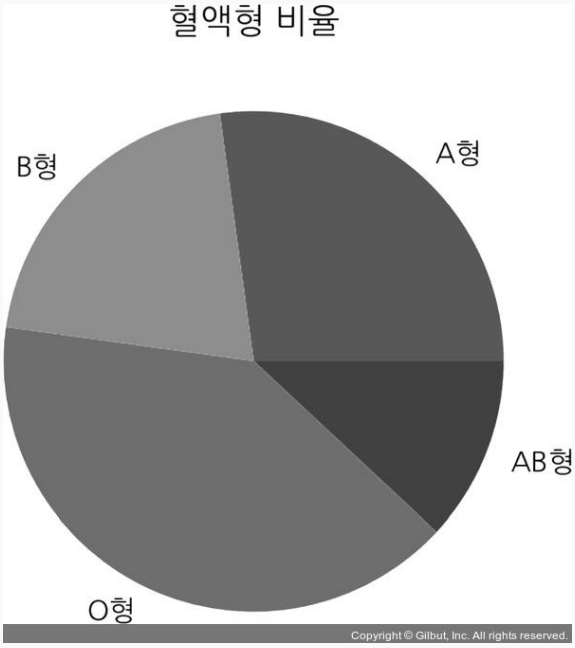


라벨 표시하기

앞의 코드를 실행하면 원그래프가 잘 나옵니다. 그런데 범례가 없으니 각 영역이 무엇을 의미하는지 알기가 어렵네요. 각 영역이 무엇을 나타내는지 원그래프에 라벨을 붙여 봅시다. 원그래프에서 라벨은 `labels` 옵션으로 붙일 수 있습니다. `plt.pie()`에서는 한 번에 그리는 영역이 여러 개이므로 앞에서 나온 `label` 옵션과 달리 뒤에 s가 붙은 `labels`임에 유의하세요. 라벨을 붙이려는 데이터가 4개이므로 리스트로 표현해야죠. 차례대로 리스트 안에 넣어 `labels=['A형', 'B형', 'O형', 'AB형']` 형태로 지정하면 됩니다. 그리고 한글이 나오므로 한글 글꼴도 설정합니다.

```
b_type = [25, 19, 37, 11]
plt.rc('font', family='Malgun Gothic') # macOS에서는 'AppleGothic'
plt.pie(b_type, labels=['A형', 'B형', 'O형', 'AB형'])
plt.show()
```

실행결과



이제 원그래프 영역별로 라벨이 표시됩니다. `labels`에 옵션 값을 넣을 때 앞에서처럼 리스트로 직접 넣어도 되지만, 변수를 사용해도 됩니다. 다음 코드처럼 영역 이름을 `b_name`이란 리스트에 저장하고 이를 `labels`에 지정합니다.

```
b_type = [25, 19, 37, 11]
b_name = ['A형', 'B형', 'O형', 'AB형']
plt.rc('font', family='Malgun Gothic')
plt.pie(b_type, labels=b_name)
plt.show()
```

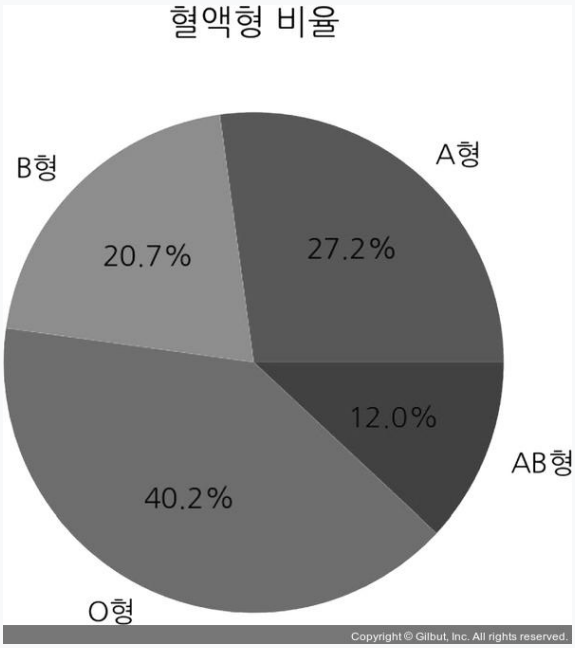
결과는 같으니 코드를 실행해서 확인해 보세요!

비율 표시하기

원그래프의 영역이 각각 크기가 달라서 어느 정도 짐작은 가지만, 정확한 비율은 알 수가 없습니다. 혈액형별 비율이 얼마인지 나온다면 그래프를 이해하기가 더 수월하겠죠? 비율은 `autopct` 옵션으로 표시합니다. 'auto percent'라는 의미입니다.

```
b_type = [25, 19, 37, 11]
b_name = ['A형', 'B형', 'O형', 'AB형']
plt.rc('font', family='Malgun Gothic')
plt.pie(b_type, labels=b_name, autopct='%1.1f%%')
plt.show()
```

실행결과



영역별로 차지하는 비율이 나옵니다. 옵션에서 값으로 넣은 `'%1.1f%%'`는 퍼센트를 소수점 몇 자리까지 나타내는가를 지정합니다. `'%1.1f%%'`는 소수점 첫째 자리까지, `'%1.4f%%'`는 소수점 넷째 자리까지 나타냅니다. 소수점 이하 자릿수를 직접 바꿔 가면서 실행해 보세요.

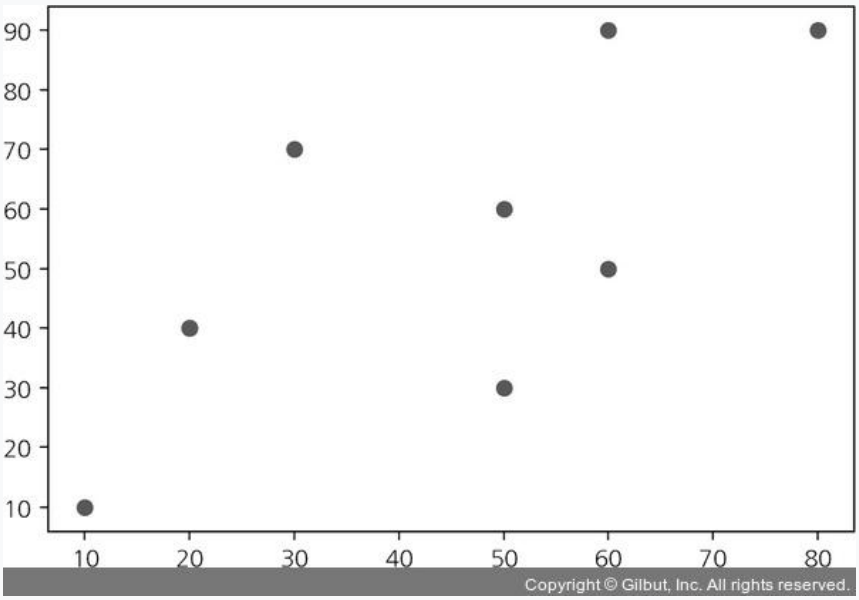
8.2.4 산점도 그리기

통계 분석에 많이 쓰는 산점도라는 그래프가 있습니다. 산점도는 x값과 y값이 만나는 지점을 점으로 찍어서 표현한 그래프입니다. 산점도는 `plt.scatter()` 명령어로 그리며, `plt.bar()`나 `plt.barh()`처럼 x축과 y축 리스트를 넣어 주면 됩니다.

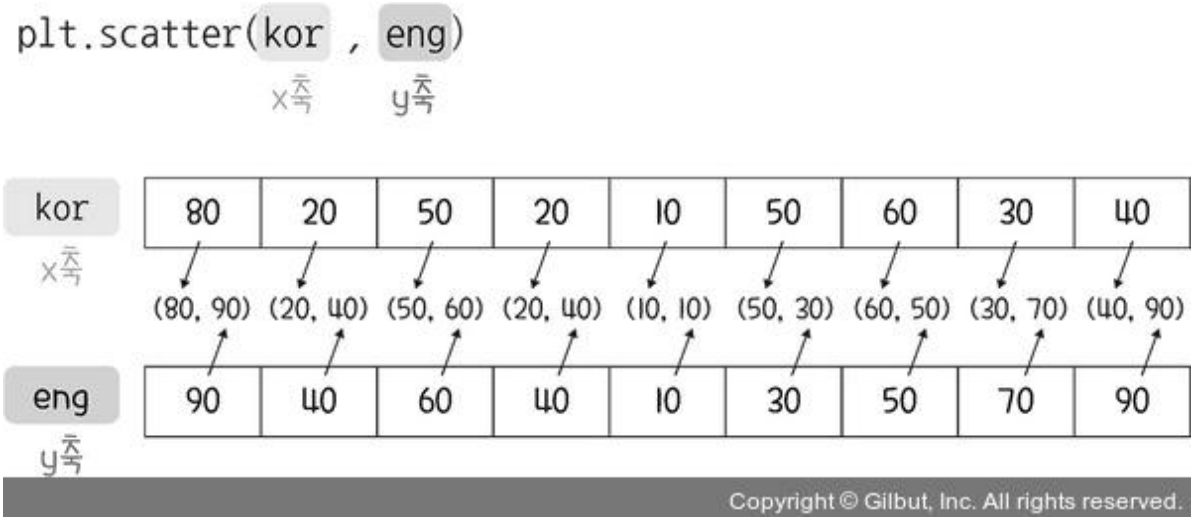
국어 점수와 영어 점수의 상관관계를 보여 주는 그래프를 그려 봅시다. 국어 점수를 저장한 리스트는 `kor`, 영어 점수를 저장한 리스트는 `eng`로 하겠습니다.

```
kor = [80, 20, 50, 20, 10, 50, 60, 30, 60]
eng = [90, 40, 60, 40, 10, 30, 50, 70, 90]
plt.scatter(kor, eng)
plt.show()
```

실행결과



실행하면 x축에 `kor` 리스트를, y축에 `eng` 리스트를 넣어서 산점도를 그립니다. `kor` 리스트의 첫 번째 요소와 `eng` 리스트의 첫 번째 요소끼리 짝을 이뤄 (80, 90)에, `kor`와 `eng`의 두 번째 요소끼리 짝을 이뤄 (20, 40)에 점을 찍습니다. 이렇게 두 리스트의 각 요소가 짝을 이뤄 산점도에 점을 찍습니다.



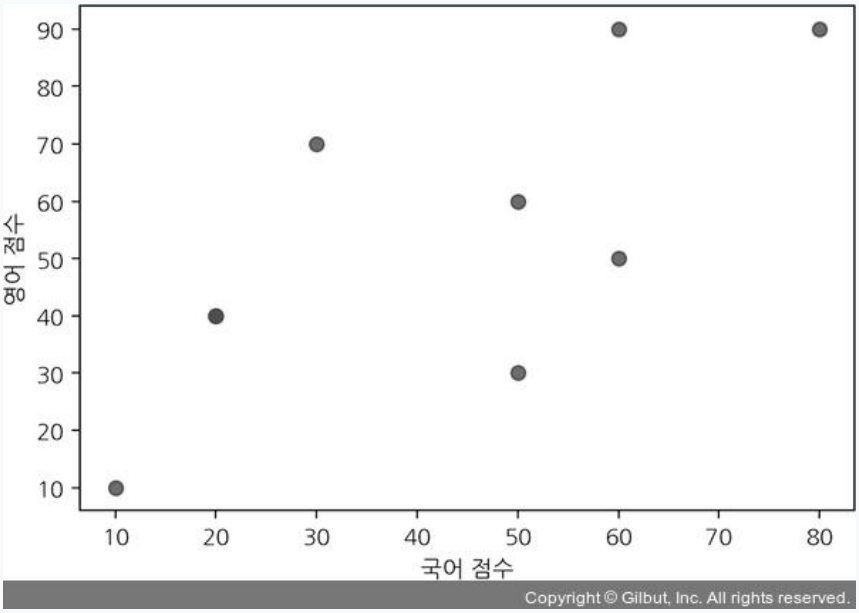
▲ 그림 8-5 산점도의 x축과 y축

색 바꾸고 투명도 설정하기

이번에는 산점도를 꾸며 보겠습니다. 먼저 점의 색을 바꾸고 투명도를 설정합니다. 투명도는 색의 '진한 정도'로 보면 됩니다. 점의 색은 다른 그래프와 마찬가지로 `color` 옵션을 사용하면 됩니다. 투명도는 `alpha` 옵션으로 설정합니다. x축 이름과 y축 이름도 추가합시다.

```
kor = [80, 20, 50, 20, 10, 50, 60, 30, 60]
eng = [90, 40, 60, 40, 10, 30, 50, 70, 90]
plt.rc('font', family='Malgun Gothic')
plt.scatter(kor, eng, color='red', alpha=0.7)
plt.xlabel('국어 점수')
plt.ylabel('영어 점수')
plt.show()
```

실행결과



점의 색은 `red`로 하되 `alpha`로 투명도를 1보다 낮게 설정해서 조금 연한 빨간색으로 바꿨습니다. 투명도를 0.5, 0.3 등으로 바꾸고 투명도 변화를 확인해 보세요. 축 이름이 한글이라서 한글 글꼴 설정도 넣었습니다. x축과 y축 이름도 추가했고요.

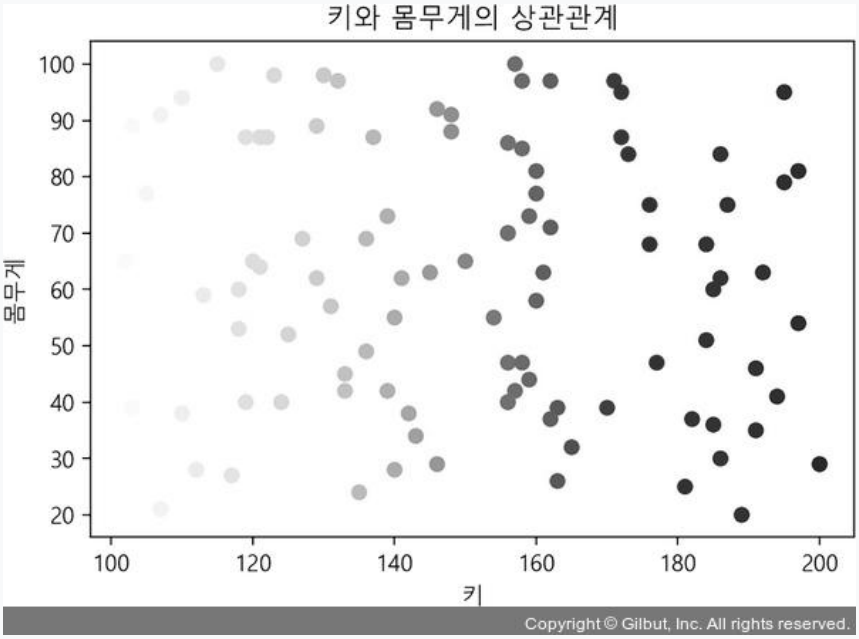
컬러맵 적용하기

이번에는 데이터를 바꿔 키와 몸무게의 상관관계를 표현하는 산점도를 그려 보겠습니다. 100명의 데이터를 표시하는데, 숫자 100개를 직접 입력하긴 어려우므로 `random` 라이브러리를 사용해 무작위로 키와 몸무게 100개를 만듭니다. 키를 저장하는 리스트는 `height`, 몸무게를 저장하는 리스트는 `weight`로 하고, 반복문을 활용해 각 리스트에 숫자 100개를 넣습니다. 그리고 컬러맵(color map)을 사용해 산점도를 다채롭게 표현합니다. 컬러맵을 사용하려면 2가지 옵션을 추가해야 합니다. `cmap` 옵션으로 사용할 컬러맵의 이름을 지정하고, `c` 옵션으로 컬러맵을 표현할 기준이 되는 축을 설정합니다. 코드를 봅시다.

```
import random

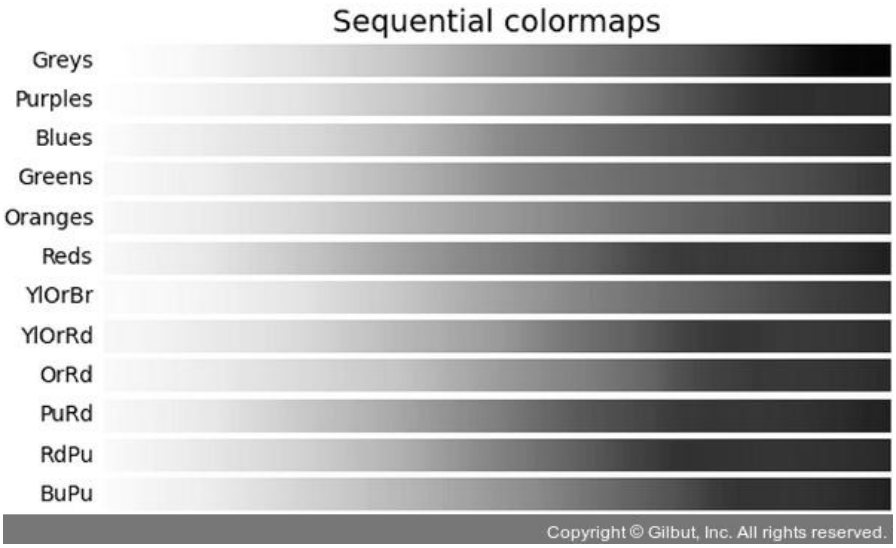
height, weight = [], []
for i in range(100):
    height.append(random.randint(100, 200))
    weight.append(random.randint(20, 100))
plt.rc('font', family='Malgun Gothic')
plt.title('키와 몸무게의 상관관계')
plt.scatter(height, weight, c=height, cmap='RdPu')
plt.xlabel('키')
plt.ylabel('몸무게')
plt.show()
```

실행결과



키와 몸무게를 저장할 `height`, `weight`를 빈 리스트로 한 번에 만듭니다. 5장에서 배웠듯이 여러 변수에 값을 한 번에 넣어 정의할 수 있습니다. 리스트도 마찬가지고요. 다음으로 무작위 값의 키와 몸무게를 리스트에 추가하는 코드를 반복문에 넣었습니다. 키는 100에서 200 사이로, 몸무게는 20부터 100 사이 값이 나오도록 설정했습니다. `plt.scatter()`의 `c` 옵션에서 색상 기준을 `height`로 지정했기 때문에 키가 작을수록 연하게, 클수록 점의 색이 진하게 나타납니다. `c` 옵션의 값을 `weight`로 바꾸면 몸무게에 따라 점의 색상 농도가 다르게 나타납니다. `c` 옵션을 변경해서 색상 변화를 확인해 보세요. 컬러맵은 `RdPu`를 지정했습니다.

TIP `matplotlib`의 컬러맵은 여러 범주로 나뉘고 범주마다 더 많은 하위 컬러맵이 있습니다. https://matplotlib.org/tutorials/colors/colormaps.html 에 가면 다양한 컬러맵을 볼 수 있습니다. 컬러맵 왼쪽에 있는 영단어가 해당 컬러맵의 이름입니다. 원하는 컬러맵을 넣어서 코드를 실행해 보세요.

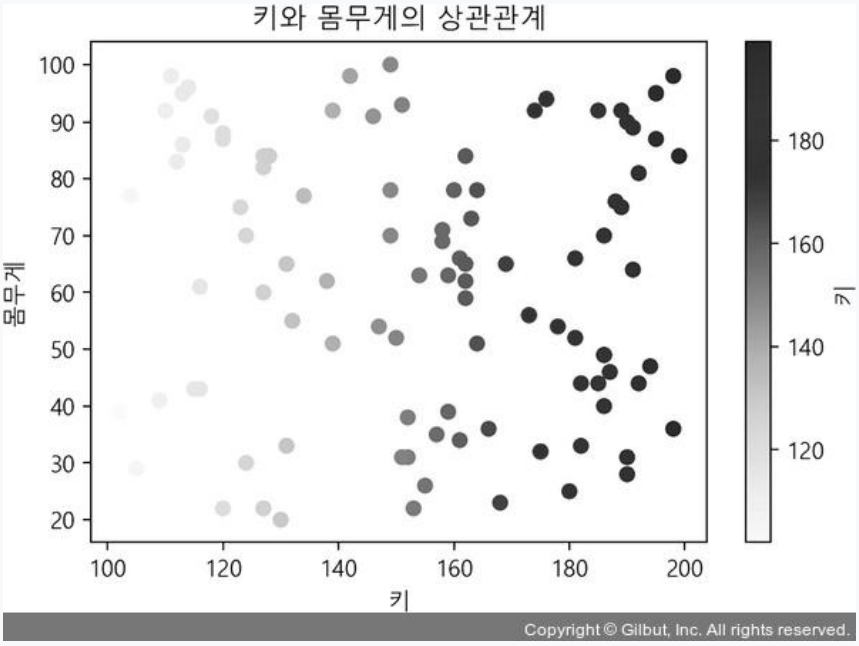


▲ 그림 8-6 컬러맵

`cmap`을 사용할 때 `plt.colorbar()` 명령어를 추가하면 그래프 옆에 숫자별 색상값을 나타내는 컬러바를 보여 줍니다.

```
…
plt.scatter(height, weight, c=height, cmap='RdPu')
plt.colorbar(label='키')
```

실행결과



산점도 옆에 컬러바가 생깁니다. `plt.colorbar()`에 `label` 옵션을 추가해 컬러바에 라벨도 붙였습니다. 실행결과를 보면 컬러바 옆에 키라고 적혀 있습니다. 컬러바가 있으니 색상이 의미하는 바를 좀 더 직관적으로 알 수 있습니다.

점의 크기 조절하기

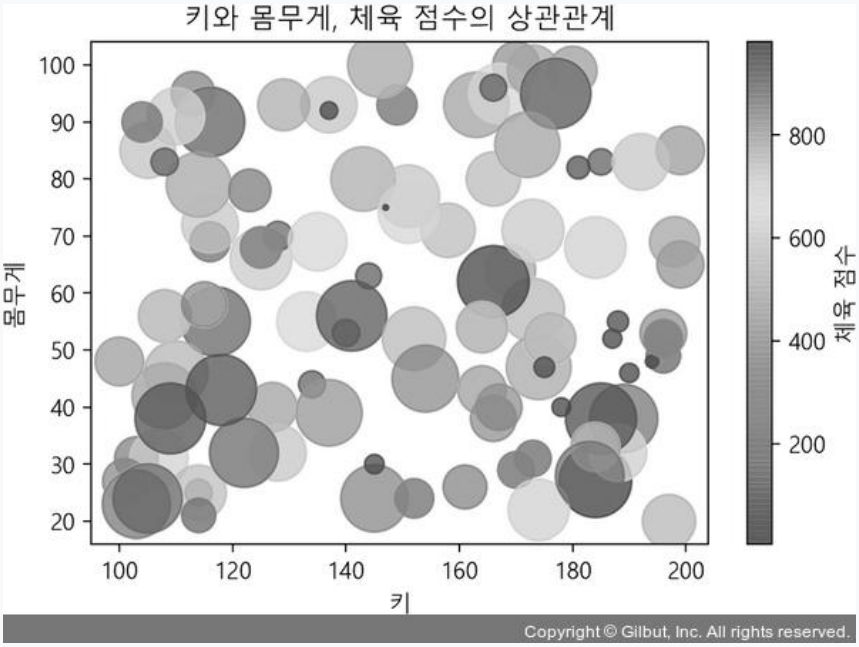
다른 그래프는 항목을 2개까지 표현할 수 있지만, 산점도는 세 항목도 한 번에 표현할 수 있습니다. x축, y축으로 두 항목을, 점의 크기로 세 번째 항목을 표현합니다. 점의 크기는 `plt.scatter()`에 `s` 옵션으로 추가해 조절합니다.

키와 몸무게가 체육 점수에 영향을 미치는지 알아봅시다. 이를 위해 키와 몸무게, 체육 점수의 상관관계를 살펴봅시다. 앞의 코드에 체육 점수를 의미하는 `score` 리스트를 추가하고 점의 크기를 `score`로 지정합니다. 체육 점수는 점의 크기가 뚜렷이 차이나도록 1부터 1000 사이의 무작위 값을 넣겠습니다.

```
height, weight, score = [], [], []
for i in range(100):
    height.append(random.randint(100, 200))
    weight.append(random.randint(20, 100))
    score.append(random.randint(1, 1000))

plt.rc('font', family='Malgun Gothic')
plt.title('키와 몸무게, 체육 점수의 상관관계')
plt.scatter(height, weight, s=score, c=score, cmap='rainbow', alpha=0.7)
plt.xlabel('키')
plt.ylabel('몸무게')
plt.colorbar(label='체육 점수')
plt.show()
```

실 행 결 과

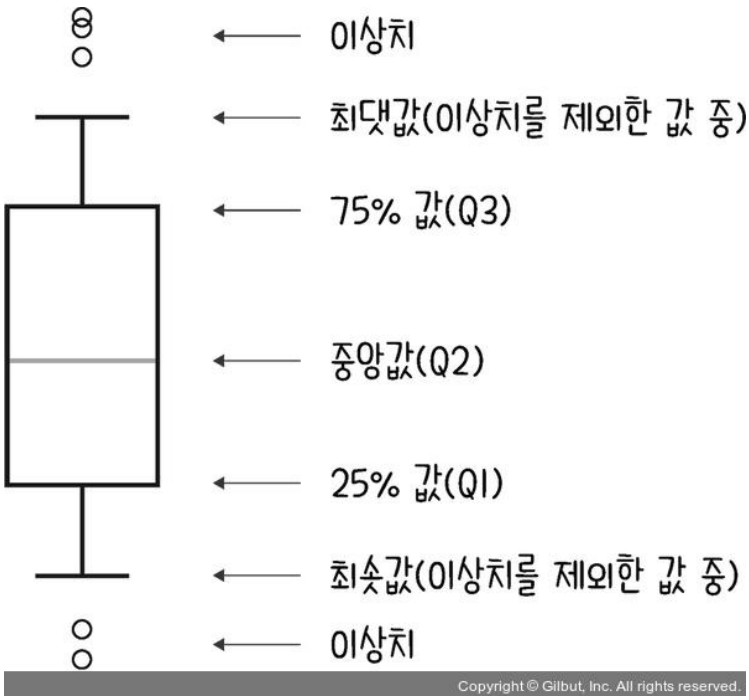


`score` 개수는 `height`와 `weight` 개수에 맞춰야 하므로 반복문 내부에 `score.append(random.randint(1, 1000))`을 추가해 무작위 값 100개를 `score`에 저장합니다. 그리고 `s` 옵션에 `score`를 지정해 체육 점수를 기준으로 점의 크기가 조절되게 합니다. 컬러맵의 기준은 체육 점수로 지정해(`c=score`) 체육 점수에 따라 점의 색이 변하게 합니다. 컬러맵은 `rainbow`로 설정하고 컬러바의 라벨도 키에서 체육 점수로 변경합니다.

결과 화면에 점의 크기가 다양하게 나타납니다. 지금은 무작위 값이라 경향성을 찾기 어렵지만, 실제 상관관계가 있는 세 항목을 넣으면 제대로 된 결괏값을 얻을 수 있습니다.

8.2.5 상자 수염 그림 그리기

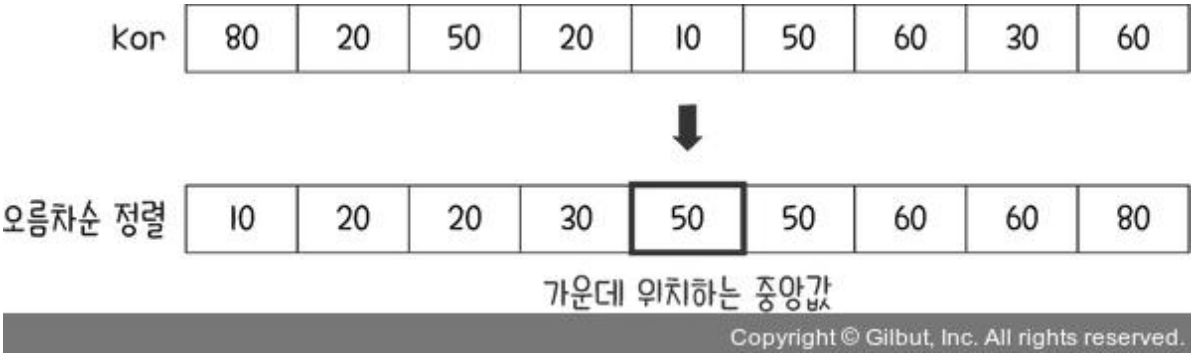
상자 수염 그림도 그려 봅시다. 상자 수염 그림(상자 그림)이란 데이터의 최솟값, 25% 값(Q1), 중앙값(Q2), 75% 값(Q3), 최댓값을 상자로 표현한 그래프입니다. 다음 그림처럼 최솟값과 최댓값을 세로로 잇는 선 위에 25% 값과 75% 값을 테두리로 하는 상자를 그립니다. 그리고 그 안에 중앙값을 표현하죠. 이때 최댓값과 최솟값은 이상치를 제외한 값 중 최댓값과 최솟값을 의미합니다.



▲ 그림 8-7 상자 수염 그림

중앙값은 평균값과는 다른 개념입니다. 평균은 모든 요소의 합을 개수로 나눈 값이죠. 평균값은 원본 데이터에 없을 수도 있습니다. 예를 들어, 국어 점수를 저장하는 `kor` 리스트에 `[80, 20, 50, 20, 10, 50, 60, 30, 60]`이 있습니다. 국어 점수의 평균값은 (80 + 20 + 50 + 20 + 10 + 50 + 60 + 30 + 60)/9 = 42.2222...로, 이 점수는 원본 데이터에 없습니다.

중앙값은 원본 데이터에서 중앙에 위치한 값입니다. `kor` 리스트를 오름차순으로 정렬하면 `[10, 20, 20, 30, 50, 50, 60, 60, 80]`이죠. 여기에서 가운데 위치하는 값은 다섯 번째에 있는 `50`입니다. `50`이 바로 `kor`의 중앙값입니다. 따라서 중앙값은 반드시 원본 데이터에 있습니다.



▲ 그림 8-8 국어 점수의 중앙값

그럼 국어 점수의 25% 값과 75% 값도 구해 보죠. 25% 값은 상위 25%에 있는 값을 의미합니다. 총 9개의 숫자가 있으니 (데이터 개수 9 × 0.25)번째에 위치한 값입니다. 계산하면 2.25가 나오네요. 반올림하면 2가 됩니다. 국어 점수에서 두 번째에 있는 값은 20이죠. 따라서 25% 값은 20이 됩니다.

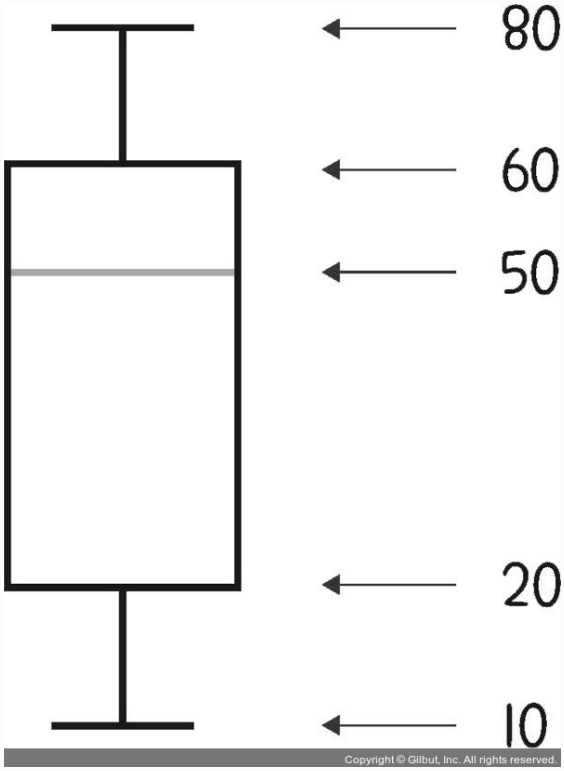
75% 값은 앞에서 75%에 있는 값을 의미합니다. 데이터 개수 9에 0.75를 곱하면 6.75가 나오고 반올림하면 7이 되죠. 국어 점수에서 일곱 번째에 있는 데이터는 60이므로 75% 값은 60이 됩니다.

NOTE 이상치

최댓값과 최솟값은 이상치를 제외한 값 중 가장 큰 값과 작은 값으로 각각 정합니다. 그럼 이상치란 무엇일까요? 이상치(outlier)란 표본 데이터 범위에서 과하게 벗어난 값을 의미합니다. 상자 수염 그래프에서 이상치를 정하는 기준은 상자의 길이를 활용합니다. 상자의 길이를 IQR이라고 하는데 IQR은 Q3 - Q1으로 구할 수 있습니다. 따라서 국어 점수 데이터에서의 IQR은 60 - 20인 40이 됩니다.

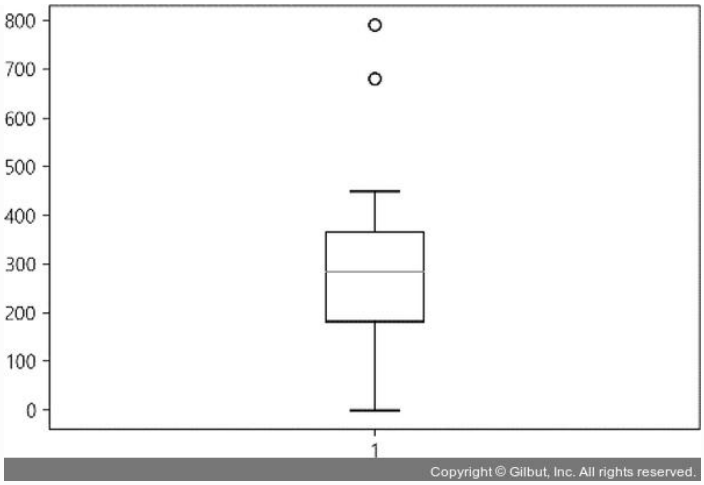
최댓값은 Q3부터 (Q3 + 1.5 × IQR) 범위에서 가장 큰 값이고, 최솟값은 (Q1 - 1.5 × IQR)부터 Q1 범위에서 가장 작은 값입니다. 그리고 이 값보다 더 크거나 작은 값은 이상치가 됩니다.

국어 점수 데이터에서 IQR은 40이고, Q1은 20, Q3는 60입니다. 따라서 60~120(60 + 1.5 × 40) 범위에서 가장 큰 값인 80이 최댓값, -40(20 - 1.5 × 40)~20 범위에서 가장 작은 값인 10이 최솟값이 됩니다. 국어 점수에는 120보다 크거나 -40보다 작은 값은 없으므로 이상치는 없습니다. 이를 상자 수염 그림으로 그리면 다음과 같은 모양이 됩니다.



▲ 그림 8-9 국어 점수의 상자 수염 그림

만약 이상치가 있다면 상자 수염 그림은 다음처럼 그려집니다.



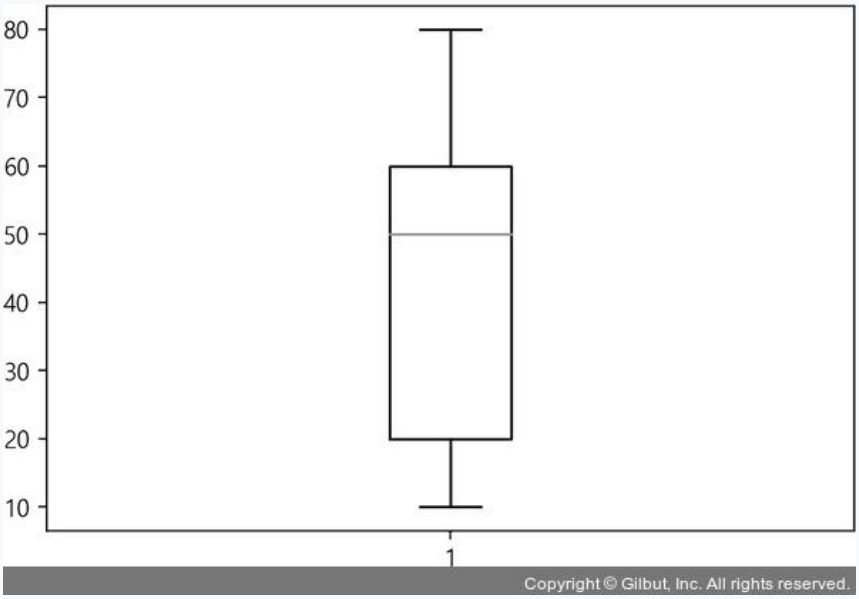
▲ 그림 8-10 이상치가 표시된 상자 수염 그림

그래프에 보이는 동그라미는 이상치를 나타냅니다. 그래프를 보면 대다수의 데이터는 0과 500 사이에 있는데, 이 범위를 벗어난 700에서 800 사이에 값이 있다는 의미로 해석하면 됩니다.

국어 점수의 분포를 상자 수염 그림으로 표현해 봅시다. 상자 수염 그림을 그리는 명령어는 `plt.boxplot()`입니다. 여기에 데이터를 넣으면 자동으로 상자 수염 그림을 그립니다.

```
kor = [80, 20, 50, 20, 10, 50, 60, 30, 60]
plt.boxplot(kor)
plt.show()
```

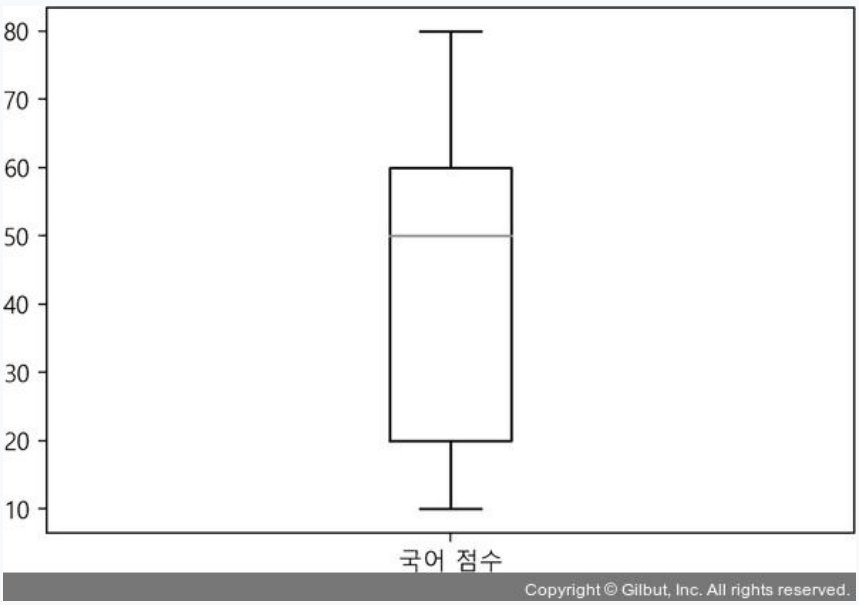
실행결과



상자 수염 그림이 그려지는데, x축이 1로 나옵니다. 상자 수염 그림에서 x축은 데이터의 라벨을 나타냅니다. 라벨은 지정하지 않으면 1을 기본값으로 표시합니다. 이를 '국어 점수'로 바꿔 봅시다. 라벨을 지정하려면 `plt.boxplot()`에 `labels` 옵션을 추가합니다.

```
kor = [80, 20, 50, 20, 10, 50, 60, 30, 60]
plt.boxplot(kor, labels=['국어 점수'])
plt.show()
```

실행결과



`labels` 옵션에는 원그래프에서 영역에 라벨을 붙일 때처럼 리스트 형태로 값을 넣습니다. 그렇지 않으면 오류가 발생합니다.

이외에도 다양한 옵션과 명령어가 있지만, 입문 단계에서는 이 정도로 충분합니다.

1분 퀴즈

2 다음 코드에 대한 설명으로 틀린 것을 고르세요.

```
import matplotlib.pyplot as plt

data = [92, 63, 8, 27, 39, 84, 76, 18, 46]
plt.plot(range(9), data)
plt.show()
```

① `plt.plot()`을 `plt.bar()`로 변경하면 막대그래프가 그려진다.

② `plt.plot()`을 `plt.barh()`로 변경하면 가로형 막대그래프가 그려진다.

③ `plt.plot()`을 `plt.hist()`로 변경하면 히스토그램이 그려진다.

④ `plt.plot()`을 `plt.scatter()`로 변경하면 산점도가 그려진다.

정답 및 해설

8.3

그래프 겹쳐 그리기

지금까지 다양한 그래프를 그려 봤습니다. 이번에는 앞에서 배운 그래프를 여러 개 겹쳐 그리는 방법을 배워 보겠습니다. 그래프를 겹쳐 그리면 여러 데이터를 비교할 수 있고 다양하게 응용할 수도 있어요.

8.3.1 꺾은선 그래프 겹쳐 그리기

신인 가수를 뽑는 오디션 프로그램에서 나온 첫째 주의 득표수와 둘째 주의 득표수를 비교하는 그래프를 그려 봅시다. 참가자는 총 5명으로 편의상 A, B, C, D, E로 하고 `singer` 리스트에 저장하겠습니다. 그리고 첫째 주의 득표수는 `week1` 리스트, 둘째 주의 득표수는 `week2` 리스트에 저장합니다. 그럼 다음과 같이 리스트 3개가 만들어집니다.

```
singer = ['A', 'B', 'C', 'D', 'E']
week1 = [42, 58, 19, 92, 84]
week2 = [53, 52, 48, 98, 73]
```

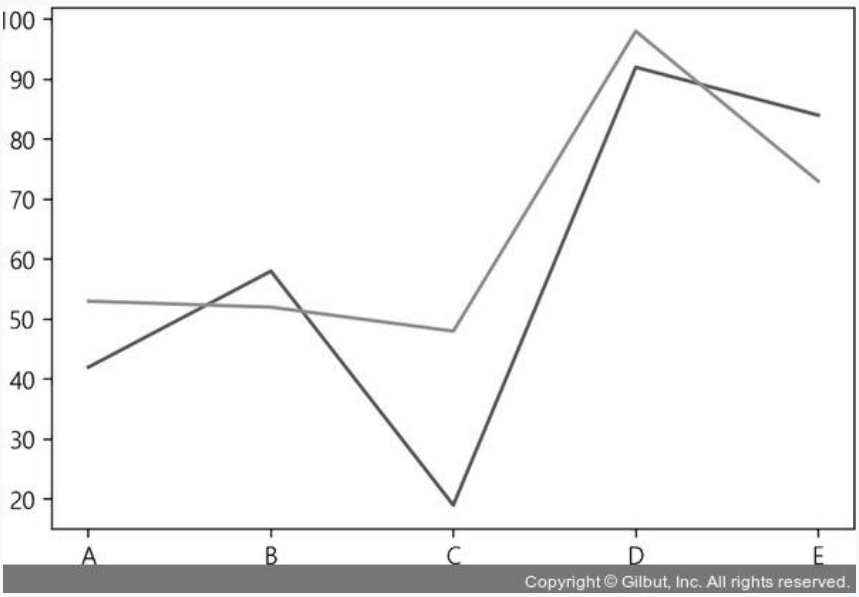
`week1`과 `week2`가 참가자별 첫째 주와 둘째 주 득표수이므로 `week1`과 `week2`로 꺾은선 그래프를 그리면 됩니다. `plt.plot()` 명령어 하나당 1개의 꺾은선 그래프가 그려지니 이를 참고해서 코드를 작성합니다.

```
import matplotlib.pyplot as plt

singer = ['A', 'B', 'C', 'D', 'E']
week1 = [42, 58, 19, 92, 84]
week2 = [53, 52, 48, 98, 73]

plt.plot(singer, week1)
plt.plot(singer, week2)
plt.show()
```

실행결과

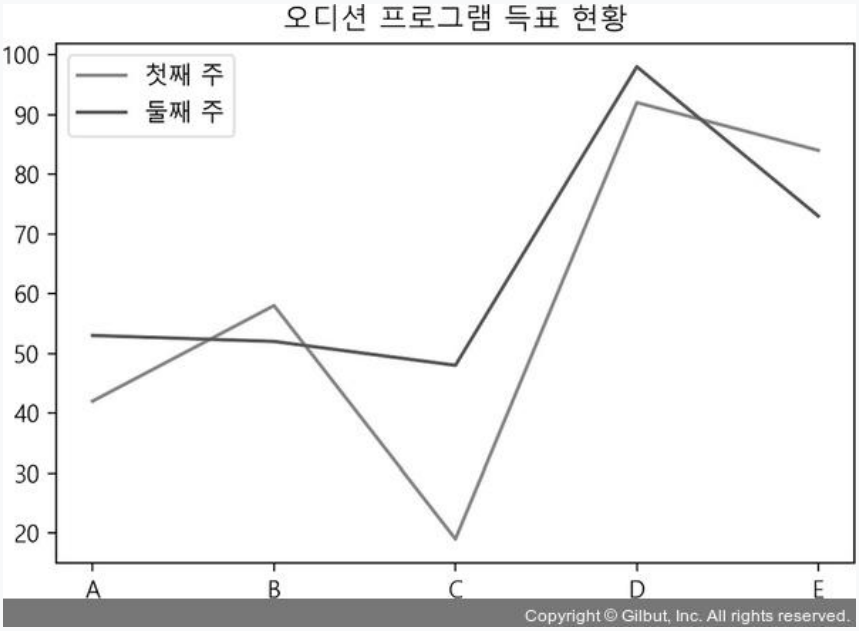


꺾은선 그래프 2개가 겹쳐서 그려집니다. `plt.plot(singer, week1)`로 참가자별 첫째 주 득표수 그래프를, `plt.plot(singer, week2)`로 참가자별 둘째 주 득표수 그래프를 그렸습니다. 이처럼 여러 개의 꺾은선 그래프를 겹쳐 그릴 때는 그래프 개수만큼 `plt.plot()`을 작성한 다음, `plt.show()`를 실행해 그래프를 한 번에 그리면 됩니다.

이번에는 그래프에 범례를 표시하고 그래프 색도 변경해 보겠습니다. 제목도 추가합시다.

```
plt.rc('font', family='Malgun Gothic')
plt.title('오디션 프로그램 득표 현황')
plt.plot(singer, week1, label='첫째 주', color='hotpink')
plt.plot(singer, week2, label='둘째 주', color='royalblue')
plt.legend()
plt.show()
```

실행결과

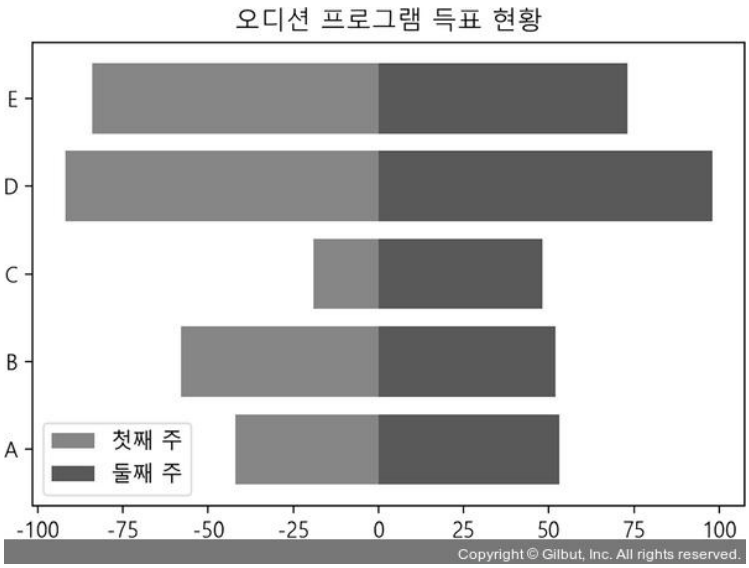


`plt.title()`로 제목을 추가하고, `label` 옵션으로 라벨을, `color` 옵션으로 그래프의 색을 지정합니다. 이렇게 `label` 옵션을 추가하면 그 아래에 `plt.legend()`를 추가해야 범례가 그래프에 표시됩니다. 제목과 범례를 추가하니 어떤 그래프인지 알아보기 쉬워졌습니다.

첫째 주보다 성적이 향상된 참가자는 A, C, D이고 성적이 떨어진 참가자는 B, E네요. 2주 연속 D가 1등, C가 꼴등을 했음이 한눈에 보입니다.

8.3.2 막대그래프로 겹쳐 그리기

막대그래프 2개를 다음 그림처럼 항아리 모양으로 그려 봅시다.



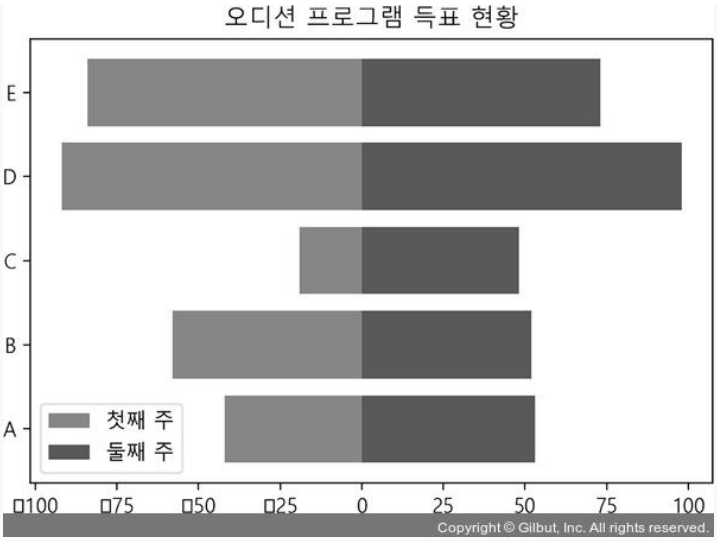
▲ 그림 8-11 항아리형 막대그래프로 그리기

가로형 막대그래프이므로 `plt.barh()` 2개로 그리면 됩니다. 그리고 x축을 보면 둘째 주 그래프는 x축이 양수지만, 첫째 주 그래프는 x축이 모두 음수입니다. 따라서 첫째 주 득표수는 음수를 만들어야 합니다. 이를 코드로 작성하면 다음과 같습니다.

```
for i in range(len(week1)):
    week1[i] = -week1[i]

plt.rc('font', family='Malgun Gothic')
plt.title('오디션 프로그램 득표 현황')
plt.barh(singer, week1, label='첫째 주', color='hotpink')
plt.barh(singer, week2, label='둘째 주', color='royalblue')
plt.legend()
plt.show()
```

실행결과


오디션 프로그램 득표 현황
E
D
C
B
A
첫째 주
둘째 주
100
75
50
25
0
25
50
75
100
Copyright © Gilbut, Inc. All rights reserved.

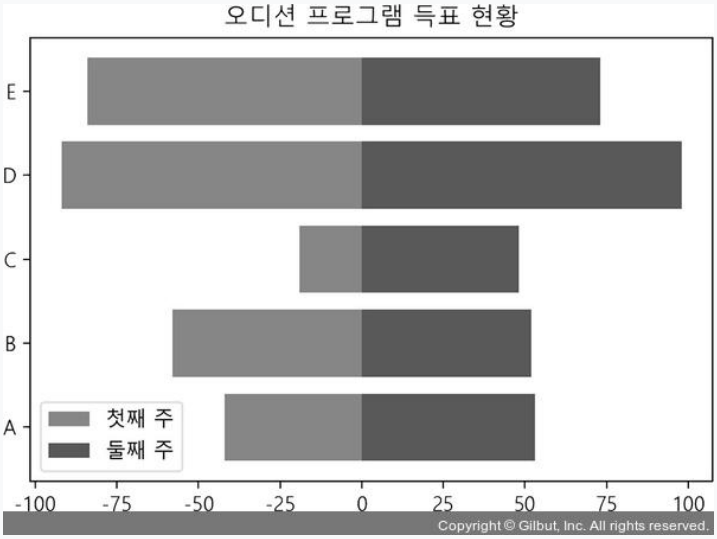
`week1` 리스트의 모든 요소를 음수로 만들어야 하므로 `week1`의 모든 요소에 `week1[i] = -week1[i]`를 수행하도록 반복문에 넣습니다. 그리고 `plt.barh()`로 가로형 막대그래프 2개를 그리면 좌우로 펼쳐진 막대그래프가 만들어집니다.

그런데 x축을 보면 값이 제대로 표시되지 않습니다. 이런 표시는 문자 인식을 제대로 못했을 때 나타납니다. 따라서 - 기호를 인식하도록 다음처럼 코드 한 줄을 더 추가해야 합니다. 코드를 추가하고 실행해 봅시다.

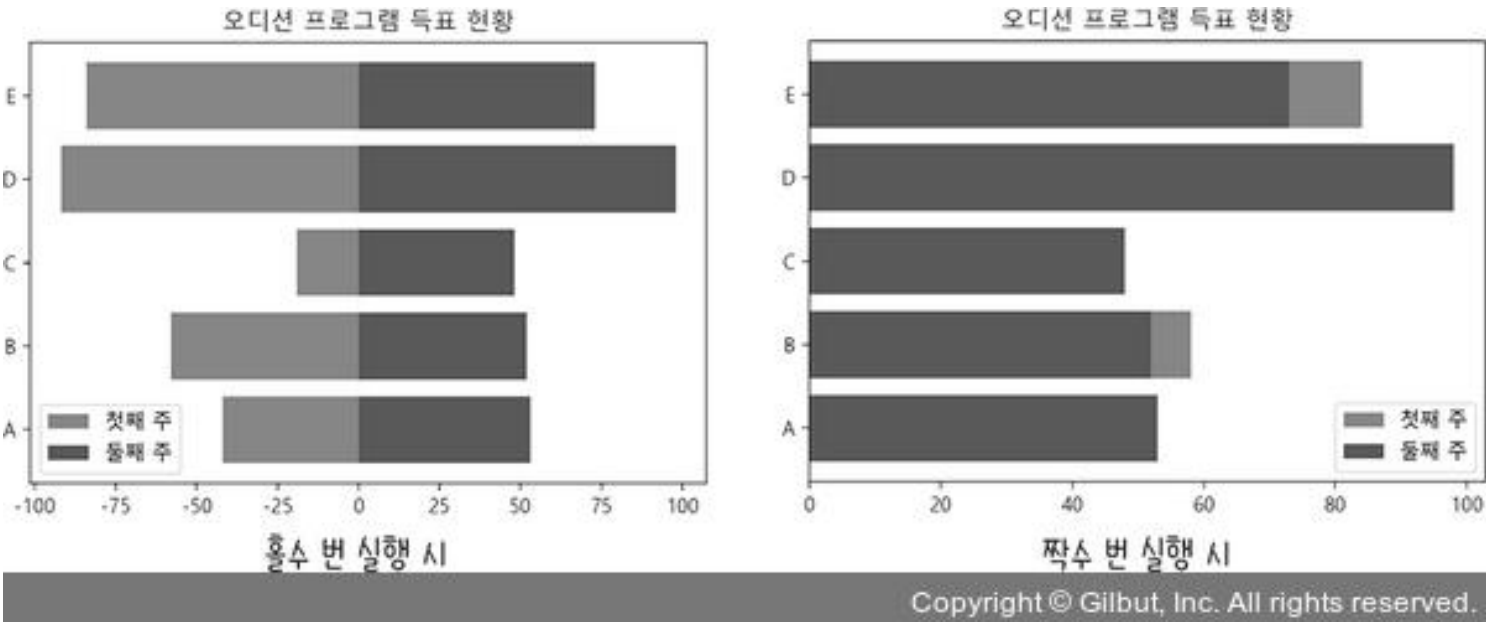
```
for i in range(len(week1)):
    week1[i] = -week1[i]

plt.rcParams['axes.unicode_minus'] = False
plt.rc('font', family='Malgun Gothic')
…
```

실행결과



- 기호가 제대로 표시됩니다. `plt.rcParams['axes.unicode_minus'] = False`는 그래프에서 - 기호가 나타나게 하는 설정 코드입니다. 이때 다음 그림의 오른쪽 부분처럼 그래프가 겹쳐서 나타나도 당황하지 마세요! 정상적인 결과입니다.



▲ 그림 8-12 실행할 때마다 달라지는 그래프 모양

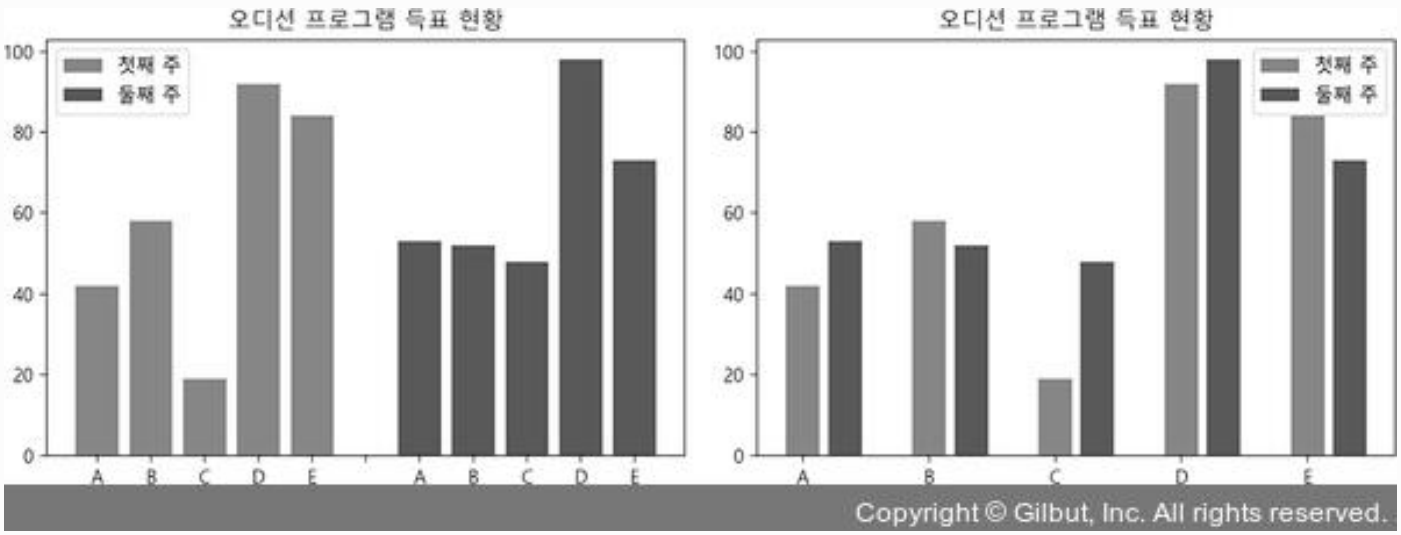
원인은 다음 코드에 있습니다.

```
for i in range(len(week1)):
    week1[i] = -week1[i]
```

셀을 한 번 실행할 때마다 `week1` 리스트의 요소를 음수로 만드는 반복문이 실행됩니다. 따라서 코드를 처음 실행하면 양수였던 값이 음수로 변환되어 왼쪽 그래프처럼 표현됩니다. 그런데 한 번 더 코드를 실행하면 음수였던 `week1`의 값들이 양수로 변환되어 오른쪽 그래프로 표현하는 것이죠. 만약 실행결과가 오른쪽 그래프처럼 나온다면 코드를 한 번 더 실행하면 됩니다.

NOTE 막대그래프를 그리는 다른 방법

여기서는 2개의 막대그래프를 항아리 모양으로 구성했지만, 다른 방식으로도 표현할 수 있습니다. 다음 그림을 봅시다.



▲ 그림 8-13 막대그래프를 겹쳐 그리는 방법

첫 번째 방법은 한 그래프 안에서 2개의 막대그래프를 좌우로 분리해 나란히 그리는 것입니다. 앞의 결과 그래프와 비교했을 때 x축이 다르다는 점이 보이나요? 가장 왼쪽의 A가 숫자 1의 위치라고 가정한다면 다음과 같이 정리할 수 있습니다.

x축 위치	1	2	3	4	5	6	7	8	9	10	11
라벨	A	B	C	D	E		A	B	C	D	E
그래프 색											

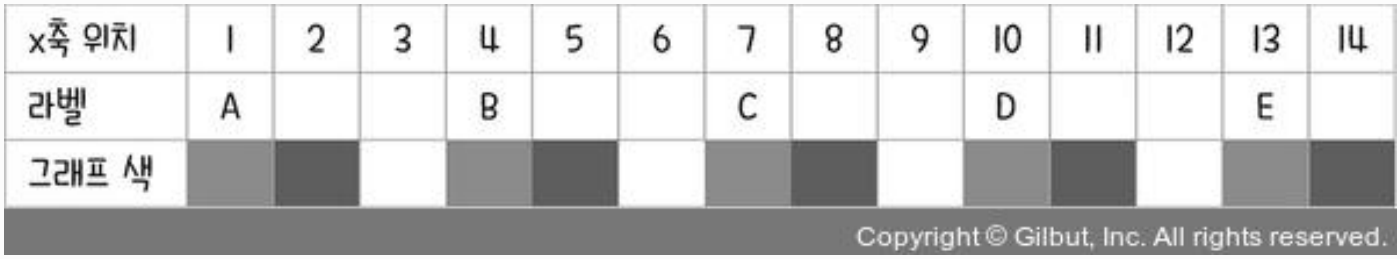
Copyright © Gilbut, Inc. All rights reserved.

분홍색 그래프의 x축은 1부터 5까지, 파란색 그래프의 x축은 7부터 11까지 표현되도록 코드를 변경해야 합니다. 그리고 x축의 숫자를 오디션 프로그램 참가자 이름으로 바꿔야죠. 이럴 때는 `plt.xticks()`를 사용합니다. `plt.xticks()`는 그래프에 표시되는 x축의 눈금 위치와 라벨을 지정하며, `plt.xticks(눈금 위치, 눈금 라벨)` 형식으로 작성하면 됩니다. 참고로, 틱(tick)은 그래프에 표시하는 눈금을 의미합니다. 코드는 다음과 같습니다.

```
plt.bar(range(1, 6), week1, label='첫째 주', color='hotpink')
plt.bar(range(7, 12), week2, label='둘째 주', color='royalblue')
plt.xticks(range(1, 12), ['A', 'B', 'C', 'D', 'E', ' ', 'A', 'B', 'C', 'D', 'E'])
```

분홍색 그래프의 x축 눈금은 1부터 5까지 연속된 숫자이므로 `range()`를 사용해 `range(1, 6)`으로 범위를 나타낼 수 있습니다. 파란색 그래프는 x축 역시 7부터 11까지 연속된 숫자이므로 `range(7, 12)`로 작성할 수 있습니다. x축 눈금 라벨은 1부터 11까지 표시해야 하니 눈금 위치를 `range(7, 12)`로 잡습니다. 그리고 원하는 라벨을 뒤에 넣습니다. 눈금 위치를 잘 따진 후, 눈금 라벨을 변경해 주면 그래프가 잘 그려집니다. 같은 형식으로 `plt.yticks()`를 사용하면 y축 라벨도 지정할 수 있습니다.

그림 8-13의 두 번째 그래프도 그려 봅시다. 이 역시 x축을 조정하면 됩니다.



Copyright © Gilbut, Inc. All rights reserved.

분홍색 그래프와 파란색 그래프가 번갈아가며 나옵니다. 간격이 둘 다 3입니다. 분홍색 그래프의 x축 눈금은 1, 4, 7, 10, 13이므로 `range(시작 숫자, 종료 숫자, 간격)` 형식을 사용해 `range(1, 14, 3)`으로 작성할 수 있습니다. 파란색 그래프는 x축 눈금이 2, 5, 8, 11, 14이므로 `range(2, 15, 3)`으로 작성할 수 있습니다. 그리고 `plt.xticks()`를 사용해 눈금 위치와 라벨을 넣습니다. 눈금이 분홍색 그래프 위치인 1, 4, 7, 10, 13에 표시되므로 `range(1, 14, 3)`으로 위치를 넣습니다. 눈금 라벨은 참가자 이름을 넣으면 되므로 `singer` 리스트를 넣습니다.

코드는 다음과 같습니다.

```
plt.bar(range(1, 14, 3), week1, label='첫째 주', color='hotpink')
plt.bar(range(2, 15, 3), week2, label='둘째 주', color='royalblue')
plt.xticks(range(1, 14, 3), singer)
```

8.3.3 산점도 겹쳐 그리기

이번에는 산점도 2개를 겹쳐서 표현해 보겠습니다. 키와 몸무게의 상관관계를 그리는데, 이번에는 그룹1과 그룹2에 각 100명씩 있다고 가정합니다. 그룹1의 키와 몸무게, 그룹2의 키와 몸무게를 산점도에 각기 다른 색상으로 표시해 봅시다.

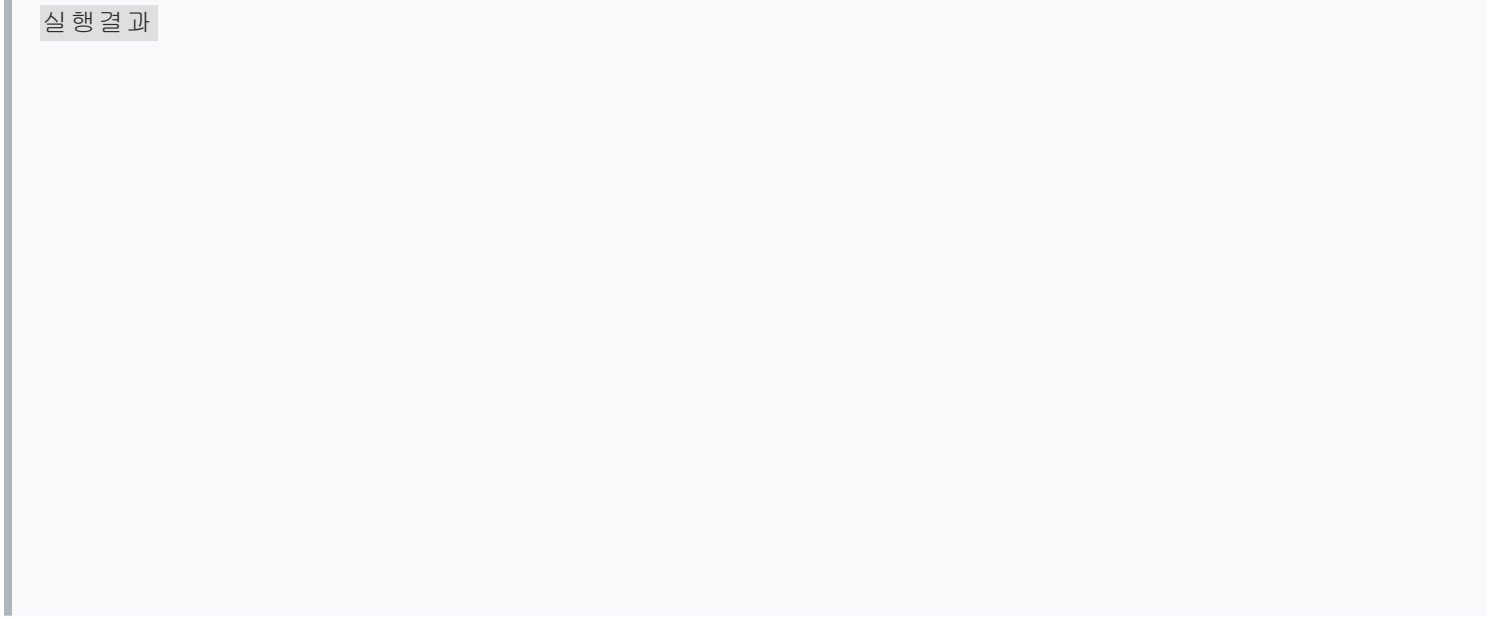
그룹1의 키와 몸무게는 `height1`, `weight1` 리스트에, 그룹2의 키와 몸무게는 `height2`, `weight2` 리스트에 저장합니다. 예시이므로 총 400개의 값을 `random` 라이브러리를 사용해 무작위로 넣습니다. 비교를 위해 그룹1과 그룹2의 키와 몸무게 범위를 다르게 설정합니다. 그룹1의 키는 140~180, 그룹2의 키는 160~200으로, 그룹1의 몸무게는 40~80, 그룹2의 몸무게는 50~100으로 설정합니다.

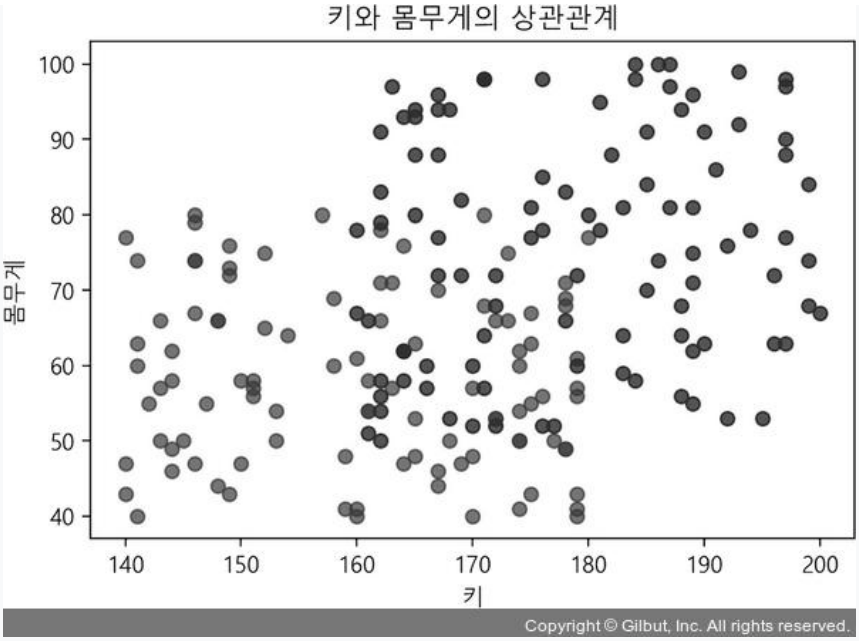
```
import random

height1, weight1, height2, weight2 = [], [], [], []
for i in range(100):
    height1.append(random.randint(140, 180))
    weight1.append(random.randint(40, 80))
    height2.append(random.randint(160, 200))
    weight2.append(random.randint(50, 100))

plt.rc('font', family='Malgun Gothic')
plt.title('키와 몸무게의 상관관계')
plt.scatter(height1, weight1, color='crimson', alpha=0.7, label='그룹1')
plt.scatter(height2, weight2, color='indigo', alpha=0.7, label='그룹2')
plt.xlabel('키')
plt.ylabel('몸무게')
plt.show()
```

실행결과





Copyright © Gilbut, Inc. All rights reserved.

그룹1과 그룹2에 각각 100명씩 있으므로 키와 몸무게를 리스트에 추가하는 명령어를 반복문으로 100번 반복합니다. 산점도를 2개 그려야 하므로 `plt.scatter()`를 두 번 사용합니다. 그룹1의 산점도는 x축을 `height1`으로, y축을 `weight1`으로 하고, 색상은 `crimson`(진홍색), 투명도는 `0.7`로 설정합니다. 그룹2의 산점도는 `height2`와 `weight2`를 각각 x축, y축으로 하고, 색상을 `indigo`(남색), 투명도는 `0.7`로 설정합니다.

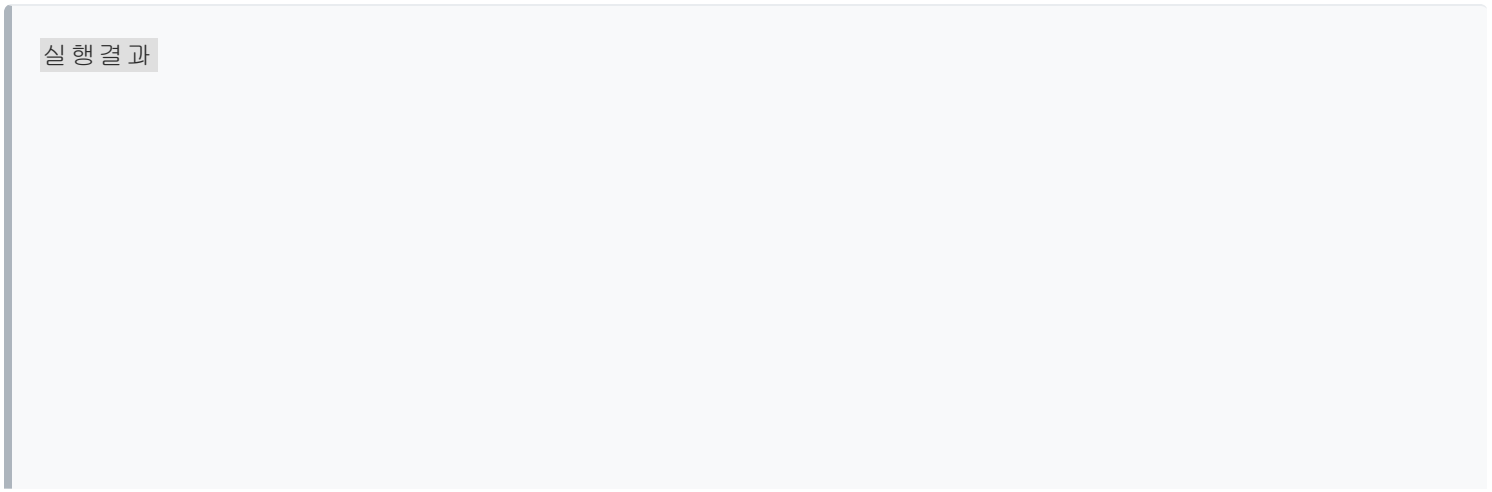
그룹1과 그룹2의 데이터 범위가 다른 것이 보이나요? 대체적으로 진홍색의 그룹1은 그래프의 왼쪽 아래에, 남색의 그룹2는 그래프의 오른쪽 위에 위치합니다. 그리고 가운데 영역에 그룹1과 그룹2가 겹치는 구간이 있고요. 이처럼 산점도는 데이터의 분포를 비교할 때 용이합니다.

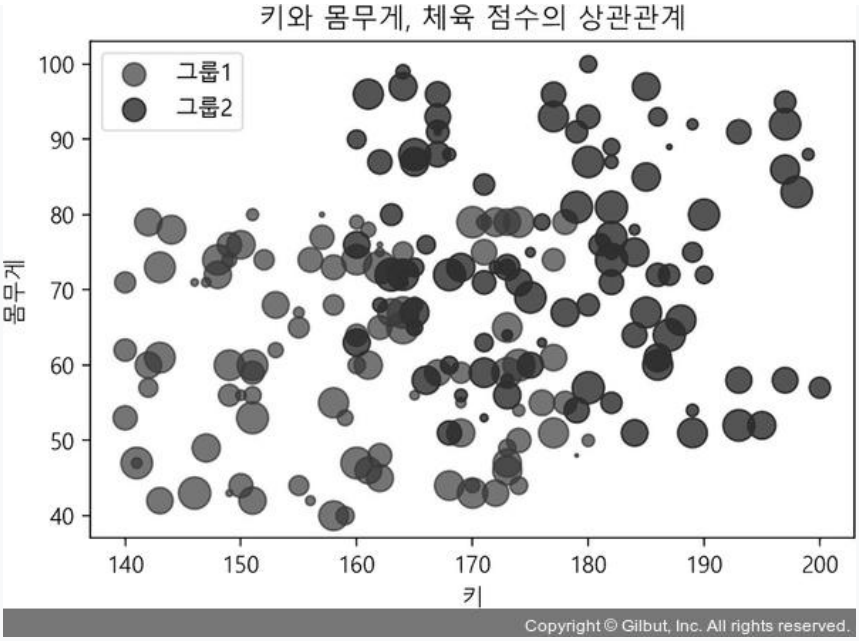
이번에는 체육 점수를 추가해 점의 크기까지 설정해 보겠습니다. 그룹1의 체육 점수는 `score1`, 그룹2의 체육 점수는 `score2`로 합니다. 그리고 두 그룹 모두 체육 점수 범위는 0~200 사이로 합니다.

```
height1, weight1, score1, height2, weight2, score2 = [], [], [], [], [], []
for i in range(100):
    height1.append(random.randint(140, 180))
    weight1.append(random.randint(40, 80))
    score1.append(random.randint(0, 200))
    height2.append(random.randint(160, 200))
    weight2.append(random.randint(50, 100))
    score2.append(random.randint(0, 200))

plt.rc('font', family='Malgun Gothic')
plt.title('키와 몸무게, 체육 점수의 상관관계')
plt.scatter(height1, weight1, s=score1, color='crimson', alpha=0.7, label='그룹1')
plt.scatter(height2, weight2, s=score2, color='indigo', alpha=0.7, label='그룹2')
plt.xlabel('키')
plt.ylabel('몸무게')
plt.legend()
plt.show()
```

실행결과





산점도를 그릴 때 `s` 옵션을 추가하니 점의 크기가 달라집니다. 이처럼 산점도도 겹쳐서 비교 그래프를 그릴 수 있습니다.

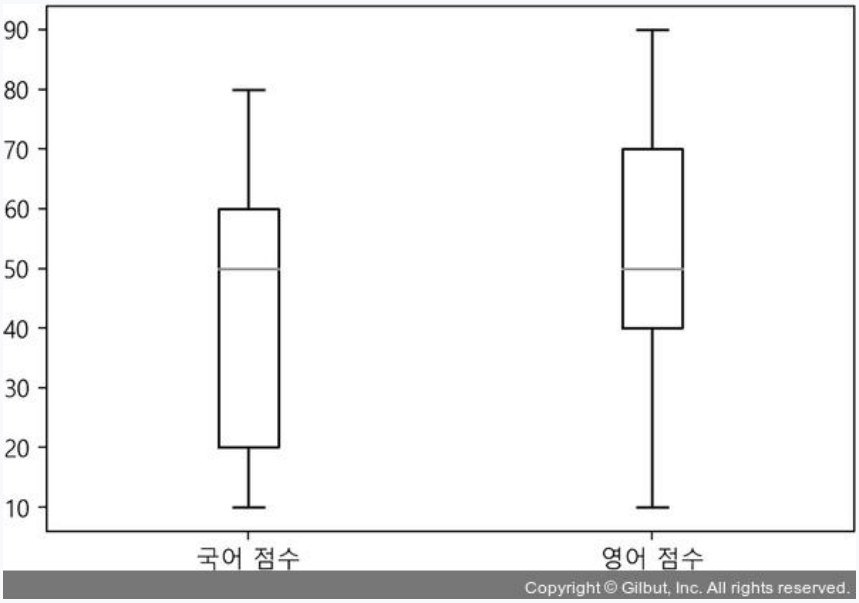
8.3.4 상자 수염 그림 여러 개 그리기

이번에는 상자 수염 그림 2개를 그려 봅시다. 영어 점수 리스트를 추가해 국어 점수와 영어 점수를 각각 상자 수염 그림으로 그립니다. 상자 수염 그림은 다른 그래프와 달리 그래프를 그리는 명령어를 여러 개 사용하지 않고, `plt.boxplot()`의 데이터에 이차원 리스트를 넣으면 됩니다. 다음 코드를 봅시다.

```
kor = [80, 20, 50, 20, 10, 50, 60, 30, 60]
eng = [90, 40, 60, 40, 10, 30, 50, 70, 90]

plt.boxplot([kor, eng], labels=['국어 점수', '영어 점수'])
plt.show()
```

실행결과



두 리스트를 묶어 이차원 리스트 `[kor, eng]`로 만들었습니다. 이차원 리스트의 첫 번째 요소는 `kor`, 두 번째 요소는 `eng`입니다. 이차원 리스트를 넣은 `plt.boxplot()`을 실행하면 이차원 리스트의 요소(일차원 리스트)별로 상자 수염 그림을 그리고 한 그래프 안에 나타냅니다. 그리고 `labels` 옵션으로 넣은 각 그래프의 라벨이 표시됩니다.

그래프를 보니 국어 점수와 영어 점수의 중앙값이 같네요. 하지만 국어 점수의 상자는 중앙값보다 아래쪽이 길고 영어 점수의 상자는 중앙값보다 위쪽이 깁니다. 따라서 국어 점수가 영어 점수보다 대체로 낮다고 볼 수 있습니다.

이차원 리스트 안에 더 많은 일차원 리스트를 넣으면 일차원 리스트 개수대로 상자 수염 그림이 그려집니다.

1분 퀴즈

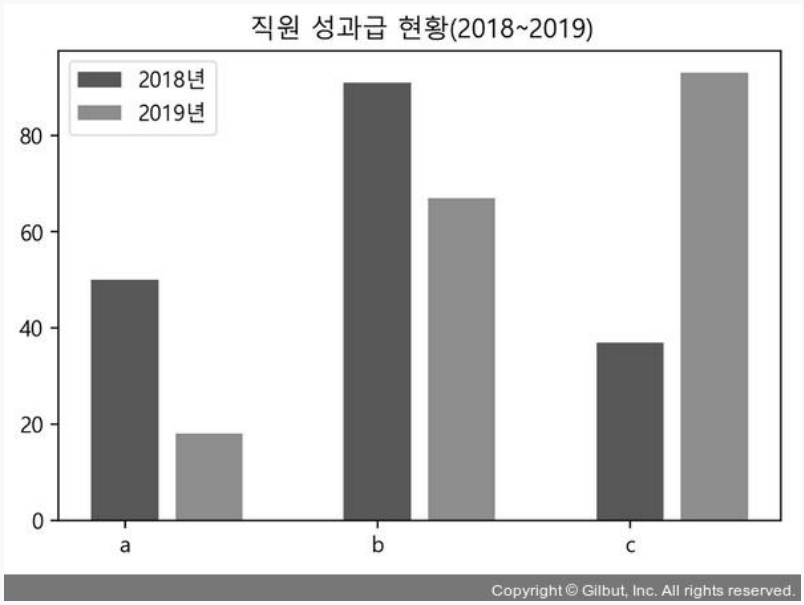
3 다음 코드를 실행한 결과, 그림과 같은 그래프가 나왔습니다. (가)와 (나)에 들어갈 코드로 알맞게 짝지어진 것을 고르세요.

```
import matplotlib.pyplot as plt

employee = ['a', 'b', 'c']
incen2018 = [50, 91, 37]
incen2019 = [18, 67, 93]

plt.rc('font', family='Malgun Gothic')
plt.title('직원 성과급 현황(2018~2019)')
plt.bar(range(1, 8, 3), incen2018, label='2018년')
plt.bar((가), incen2019, label='2019년')
plt.xticks(range(1, 8, 3), (나))
plt.legend()
plt.show()
```

실행결과



(가) / (나)

① `range(6)` / - `range(1, 8, 3)`

② `range(1, 8, 3)` / - `range(1, 8, 3)`

③ `range(1, 8, 3)` / - `employee`

④ `range(2, 9, 3)` / - `employee`

⑤ `range(2, 9, 3)` / - `label`

정답 및 해설

8

마무리

1 그래프 그리기

형식

```
import matplotlib.pyplot as plt

plt.rc('font', family='글꼴 이름')
plt.title('그래프 제목')
# 그래프 형태에 따라 알맞은 명령어 넣기
plt.show()
```

2 그래프 유형별 그리기 명령어

유형	형식	비고
꺾은선 그래프	`plt.plot([x축 리스트], [y축 리스트])`	
막대그래프	`plt.bar([x축 리스트], [y축 리스트])`	
가로형 막대그래프	`plt.barh([x축 리스트], [y축 리스트])`	
히스토그램	`plt.hist([리스트], bins=구간 개수)`	x축, y축을 구분하지 않고 하나의 리스트만 넣는다.
원그래프	`plt.pie([리스트])`	x축, y축을 구분하지 않고 하나의 리스트만 넣는다.
산점도	`plt.scatter([x축 리스트], [y축 리스트])`	
상자 수염 그림	`plt.boxplot([리스트])`	x축, y축을 구분하지 않고 하나의 리스트만 넣는다. 상자 수염 그림을 겹쳐 그릴 때는 이차원 리스트로 넣는다.

3 그래프 그리기 명령어에 추가 가능한 옵션

구분	옵션	설명
그래프 색상(그래프 공통)	`color='색상 이름'`	그래프의 색 설정하기
	`alpha=투명도`	그래프의 투명도 설정하기
라벨과 범례(그래프 공통)	`label='라벨'`	라벨 표시하기 사용 가능한 그래프: 꺾은선 그래프, 막대그래프, 산점도
	`labels=[라벨]`	사용 가능한 그래프: 원그래프, 수염 상자 그림
히스토그램	`bins=구간 개수`	히스토그램의 구간 개수 설정하기
원그래프	`autopct='%1.1%%'`	영역별 비율 자동 표시하기
산점도	`cmap='컬러맵 이름'`	컬러맵 적용하기
	`c='컬러맵을 적용할 데이터'`	컬러맵을 적용할 데이터 지정하기
	`s='점의 크기'`	점의 크기 설정하기

4 기타 명령어

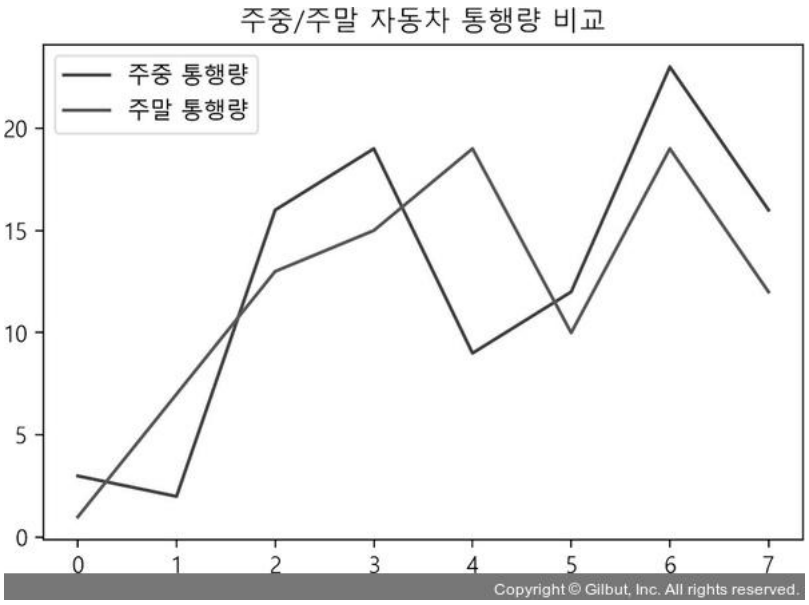
구분	명령어	설명
그래프 크기	`plt.figure(dpi=해상도)`	그래프 크기를 해상도로 설정하기
	`plt.figure(figsize=가로, 세로)`	그래프 크기를 인치 단위의 가로, 세로 길이로 설정하기
범례	`plt.legend()`	범례 표시하기
축 이름	`plt.xlabel()`	x축 이름 붙이기
	`plt.ylabel()`	y축 이름 붙이기
눈금 라벨	`plt.xticks()`	x축 눈금 라벨 설정하기
	`plt.yticks()`	y축 눈금 라벨 설정하기
컬러바	`plt.colorbar()`	산점도에 컬러바 표시하기

Self Check

다음 두 리스트는 자동차 통행량을 3시간마다 측정한 데이터입니다. weekday는 주중 자동차 통행량을, weekend는 주말 자동차 통행량을 의미합니다. 데이터를 이용해 주중과 주말 자동차 통행량을 비교하는 그래프를 다양한 방식으로 그리려고 합니다.

```
weekday = [3, 2, 16, 19, 9, 12, 23, 16]   # 주중 자동차 통행량
weekend = [1, 7, 13, 15, 19, 10, 19, 12]  # 주말 자동차 통행량
```

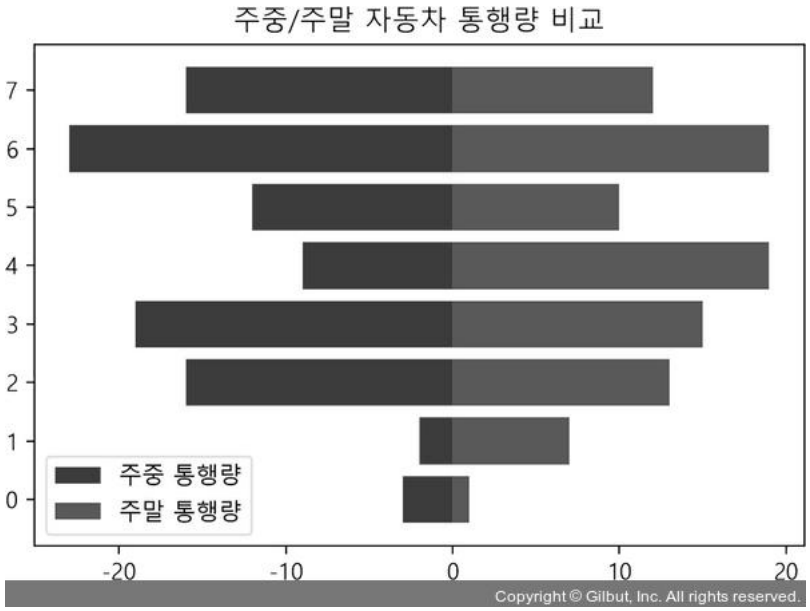
1 다음 그래프를 그려 보세요.



|조건|

- x축 눈금은 편의상 숫자 0부터 7까지로 한다.
- 주중 통행량 그래프 색상은 crimson, 주말 통행량 그래프 색상은 royalblue이다.

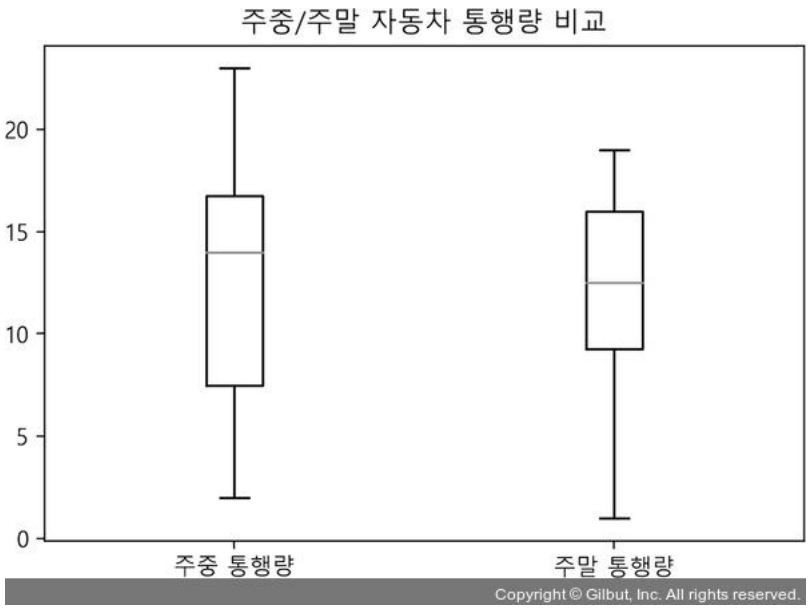
2 다음 그래프를 그려 보세요.



|조건|

- y축 눈금은 편의상 숫자 0부터 7까지로 한다.
- 주중 통행량 그래프 색상은 crimson, 주말 통행량 그래프 색상은 royalblue이다.

3 다음 그래프를 그려 보세요.



정답 및 해설